



Stock-er

Submitted In Partial Fulfillment of Requirements
For the Degree Of

Bachelor of Science (Computer Science Hons)

By

Avinashi Vora

Roll No: 31011122118

Guide

Dr. Sunita A. Yadav

S K Somaiya College

Somaiya Vidyavihar University

Vidyavihar, Mumbai - 400 077

2022-25

Somaiya Vidyavihar University

S K Somaiya College

Certificate

This is to certify that the project report on **Stock-er** is a bonafide record of the project work done by **Avinashi Vora** in the year 2024-25 under the guidance of **Dr. Sunita A. Yadav** Department of Information Technology and Computer Science in partial fulfillment of requirement for the Bachelor of Science (Computer Science Hons) degree in S K Somaiya College of Somaiya Vidyavihar University.

Dr. Sunita A. Yadav
(Guide)

Ms. Minal Dive
(Program Coordinator)

Dr. Swati Maurya
(Head of the Department)

Dr. Neetin Desai
(Dean and Professor)

Date:

Place: Mumbai-77

Somaiya Vidyavihar University

S K Somaiya College

Certificate of Approval of Examiners

This is to certify that the project report on **Stock-er** is a bonafide record of the project work done by **Avinashi Vora** in partial fulfillment of the requirement for the Bachelor of Science (Computer Science Hons.) degree in S K Somaiya College of Somaiya Vidyavihar University.

External Examiner/Expert

Internal Examiner/Guide

Date:

Place: Mumbai-77

Somaiya Vidyavihar University

S K Somaiya College

DECLARATION

I declare that this written report submission represents the work done based on my and / or others' ideas with adequately cited and referenced the original source. I also declare that I have adhered to all principles of academic honesty and integrity as I have not misinterpreted or fabricated or falsified any idea/data/fact/source/original work/ matter in my submission.

I understand that any violation of the above will be cause for disciplinary action by the college and may evoke the penal action from the sources which have not been properly cited or from whom proper permission is not sought.

Signature of the Student

Avinashi Vora

Name of the Student

31011122118

Roll No.

Date:

Place: Mumbai-77

Abstract

Stock-er is an **Inventory Management System (IMS)** designed to provide organizations with a seamless and efficient way to manage their inventory operations.

By offering structured workflows, intuitive user interactions, and real-time inventory tracking, Stock-er simplifies day-to-day tasks for businesses, ensuring accuracy and transparency in stock management.

With multi-tier access control, Stock-er enables different user roles—Admins, Managers, and Staff—to interact with inventory data according to their assigned permissions. Organizations can categorize items, generate barcodes for easy tracking, and process transactions with a streamlined checkout system.

The platform enhances operational efficiency by centralizing inventory management, ensuring role-based access, and delivering real-time updates across multiple users and departments.

Stock-er is built to serve businesses of all sizes, eliminating manual inventory errors and fostering a collaborative environment where users can track, organize, and manage stock effortlessly.

Acknowledgement

I would like to express my heartfelt gratitude to **S K Somaiya College, Somaiya Vidyavihar University** for providing me with the platform to pursue my academic goals and complete my final project. The resources and opportunities offered by the institution have been invaluable to my learning and growth.

My sincere thanks to the **Dean Dr. Neetin Desai**, for his constant encouragement and support throughout this journey. His leadership has been a guiding force in shaping my academic experience.

I am deeply grateful to **Dr. Swati Maurya**, Head of the Department of Computer Science, and **Ms. Minal Dive**, Coordinator of the Department, for their unwavering support, guidance, and encouragement throughout the duration of this project. Their mentorship has been essential in helping me navigate the challenges of this work.

A special note of thanks to my project guide, **Dr. Sunita A. Yadav**, for her continuous motivation, invaluable insights, and constructive feedback. Her dedication and support have played a pivotal role in enhancing the quality of my project and ensuring its success.

Finally, I would also like to take this opportunity to thank my **parents, sister and friends** for their constant moral support and understanding. Their belief in me has been a source of strength throughout the course of this project.

Index

Chapter No	Name of Chapter		Page No
1	Introduction		12
	1.1	Introduction	12
	1.2	Literature Review	13
	1.3	Background	15
	1.4	Objectives	17
	1.5	Purpose, Scope and Applicability	19
2	Survey of Technology		23
	2.1	Technologies	23
	2.2	Frontend Modules	25
	2.3	Backend Modules	26
3	Requirement Analysis		27
	3.1	Problem Definition	27
	3.2	Requirements Specification	28
		3.2.1 Functional Requirements	28
		3.2.2 Non-Functional Requirements	31
	3.3	SDLC Model	33
		3.3.1 Why the Iterative Model?	33
		3.3.2 Iterative Phases	34
		3.3.3 Advantages of This Approach	35
	3.4	Planning and Scheduling	36
		3.4.1 Development Phases	37
		3.4.2 Timeline Overview	38

	3.5	Software and Hardware Requirements		40
		3.5.1	Development Environment Requirements	40
		3.5.2	End-User System Requirements	42
		3.5.3	Infrastructure Requirements	43
	3.6	Preliminary Product Description		44
	3.7	Conceptual Models		45
		3.7.1	Activity Diagram	46
		3.7.2	Sequence Diagram	48
		3.7.3	Entity-Relationship Diagram (ERD)	49
		3.7.4	Use Case Diagram	50
		3.7.5	Data Flow Diagram (DFD)	51
		3.7.6	Component Diagram	52
		3.7.7	System Architecture Diagram	53
4	System Design			54
	4.1	Basic Modules		54
	4.2	Data Design		56
		4.2.1	Core Data Entities and Attributes	57
		4.2.2	Schema Characteristics	61
	4.3	Procedural Design		61
	4.4	User Interface Design		66
	4.5	Security Issues		72
	4.6	Test Case Design		74
5	Implementation and Testing			76
	5.1	Implementation Approaches		76
		5.1.1	Layered Architecture	

		5.1.2	Component-driven Development	76
		5.1.3	Cloud-connected Offline Workflow	77
		5.1.4	Versioning and Maintenance Strategy	77
	5.2	Coding Details and Coding Efficiency		78
	5.3	Testing Approaches		80
	5.4	Modifications and Improvements		84
6	Result and Discussion			87
	6.1	Test Report		87
	6.2	User Documentation		91
7	Conclusion			95
	7.1	Summary		95
	7.2	Limitations of the System		96
	7.3	Future scope		97
References				100

List of Figures and Tables

Sr No	Figure or Table No	Name of the Figure	Page No
1	Figure 3.3.2	Iterative Model	Error! Bookm ark not defined .
2	Table 3.4.2.1	Timeline Overview	39
3	Figure 3.4.2.2	Task List	39
4	Figure 3.4.2.3	Gantt Chart	40
5	Figure 3.7.1.1	Activity Diagram - 1	46
6	Figure 3.7.1.2	Activity Diagram - 2	47
5	Figure 3.7.2.1	Sequence Diagram	48
6	Figure 3.7.3.1	Entity-Relationship Diagram (ERD)	49
7	Figure 3.7.4.1	Use Case Diagram	50
8	Figure 3.7.5.1	Data Flow Diagram (DFD)	51
9	Figure 3.7.6.1	Component Diagram	52
10	Figure 3.7.7.1	System Architecture Diagram	53
11	Figure 4.4.1	Login Page	66
12	Figure 4.4.2	Invitation Code Registration Page	67

13	Figure 4.4.3	Admin Registration Page	67
14	Figure 4.4.4	Dashboard	68
15	Figure 4.4.5	Billing Page	68
16	Figure 4.4.6	Manage Category Page	69
17	Figure 4.4.7	Item Addition Page	69
18	Figure 4.4.8	Category Details	70
19	Figure 4.4.9	Browse to Add	70
20	Figure 4.4.10	Stock Addition	71
21	Figure 4.4.11	Generated Bill	71

Chapter 1: Introduction

1.1 Introduction

In today's fast-paced business environment, efficient inventory management is crucial for maintaining smooth operations and staying competitive. Whether it is tracking stock levels, organizing products, or processing sales, businesses need a structured and intuitive system to reduce errors and streamline workflow.

Stock-er is designed to meet this need—a modern Inventory Management System (IMS) that simplifies traditional inventory processes and provides a seamless digital experience. Built for businesses of all sizes, Stock-er empowers users to manage stock with ease, improve operational workflows, and maintain visibility over inventory movement in real time.

At the heart of Stock-er is a multi-tier user structure that ensures well-defined access across Admins, Managers, and Staff. Admins oversee system settings and user management, Managers supervise workflows and stock updates, while Staff handle day-to-day inventory tasks. This layered control ensures accountability and secure access at every level.

Stock-er supports a multi-tenant architecture, ensuring that each organization's data remains private and isolated while sharing a centralized infrastructure. With cloud deployment and real-time synchronization, users across devices and locations can access the latest inventory data without discrepancies or delays.

To enhance tracking, Stock-er offers built-in support for barcode generation across multiple formats, enabling organizations to assign and scan unique product barcodes for efficient stock handling, with the barcode type customizable from a variety of supported barcodes based on the use case. A dedicated billing module allows users to apply discounts — either as a percentage or flat value — and apply the taxes — as a percentage — during checkout. The resulting bills are downloaded in the selected directory of the local system, with users able to print saved receipts as needed.

The platform also prioritizes secure access, combining standard password-based login with email-based verification codes during account sign-up, user invitations, and password recovery. This approach adds an extra layer of verification without complicating the user experience.

In a world where inventory discrepancies can result in revenue loss, customer dissatisfaction, or logistical chaos, having a dependable system like Stock-er can make a tangible difference. It bridges the gap between manual record-keeping and smart automation, giving businesses a scalable tool with which they can grow.

Furthermore, Stock-er promotes operational consistency by encouraging teams to follow standardized procedures. Whether it is restocking a product, updating an item's category, or generating a bill, the platform enforces clean, traceable actions that contribute to better organizational accountability and reduced error margins.

With its role-based access, simplified workflows, barcode integration, and real-time collaboration, Stock-er addresses the prevailing obstacles of manual inventory handling. It delivers an efficient, scalable, and user-friendly solution for businesses aiming to modernize their inventory operations.

1.2 Literature Review

Inventory management has long been a foundational aspect of business operations, evolving significantly from rudimentary manual processes to sophisticated, automated systems that enhance operational accuracy and efficiency. Over the decades, a wide array of tools, methodologies, and best practices have emerged to streamline stock control, optimize inventory flow, and improve organizational performance.

Historical Perspective

Initially, inventory tracking was conducted manually, with stock levels recorded in ledgers or on paper-based logs. While this approach offered simplicity for small-scale businesses, it was prone to human error, time inefficiency, and difficulty in maintaining data consistency. As businesses grew in complexity, the need for more structured inventory tracking methods became apparent.

The emergence of digital inventory systems in the late 20th century marked a significant shift, introducing database-driven approaches that improved data reliability and retrieval speed. Soon after, the rise of Enterprise Resource Planning (ERP) systems brought integration between inventory control and broader business functions—such as finance, sales, procurement, and logistics—enabling centralized management across organizational departments.

Modern Inventory Management Solutions

In recent years, the shift towards cloud computing has redefined inventory management. Cloud-based platforms now provide businesses with greater flexibility, mobility, and scalability. Multi-tenant system architectures allow for isolated data environments within shared infrastructure, supporting concurrent use by multiple organizations without compromising privacy or performance.

Technological advancements have introduced new efficiencies into the inventory domain. Real-time synchronization ensures that inventory data remains up to date across multiple users and devices. Barcode systems have become essential for rapid item identification, tracking, and transactional accuracy. In some advanced platforms, artificial intelligence (AI) and machine learning algorithms are now employed to support demand forecasting and automate stock replenishment decisions.

Comparative Analysis of Existing Systems

The current market offers a wide range of inventory management solutions, each tailored to specific organizational needs. Traditional ERP systems provide extensive integration capabilities, often suited for large enterprises with complex operations. In contrast, lightweight Software-as-a-Service (SaaS) models are favoured by small and medium-sized businesses for their simplicity, lower setup costs, and flexibility.

However, many existing platforms present challenges in terms of usability, requiring lengthy onboarding or technical expertise. Some solutions lack adaptability for specific workflows or offer interfaces that are not user-friendly for routine tasks. Moreover, concerns about data security,

especially in cloud-based deployments, remain prevalent, with risks of unauthorized access or data leaks posing ongoing challenges.

Positioning of Stock-er in the Current Landscape

Stock-er positions itself as a modern, user-focused, and scalable inventory management system that directly addresses the limitations of traditional platforms. Designed with simplicity and efficiency in mind, it avoids the heavy overhead of conventional ERP systems while delivering critical inventory functions such as real-time updates, barcode integration, billing, and role-based user access.

The platform's cloud-native, multi-tenant architecture ensures each organization's data remains securely isolated while benefiting from shared system resources. Its real-time synchronization allows users across multiple devices or locations to work with the latest data simultaneously, eliminating the delays and inconsistencies that often plague legacy systems.

Furthermore, **Stock-er** emphasizes an intuitive user experience, reducing the learning curve for non-technical staff and supporting efficient day-to-day operations. By balancing structure with flexibility, **Stock-er** emerges as a viable alternative to both heavy ERP systems and overly simplistic tools—offering a solution that scales with the needs of modern businesses.

1.3 Background

As businesses scale in size and complexity, inventory management evolves from a routine administrative task into a critical component of operational success. It directly influences cost control, customer satisfaction, and the ability to make informed, timely decisions. However, many organizations continue to rely on outdated or fragmented inventory systems—often a mix of spreadsheets, paper records, and legacy software—that create inefficiencies, increase the risk of human error, and reduce overall visibility across the organization.

Stock-er emerged as a direct response to these pain points, aiming to redefine how organizations manage stock in the modern digital era. Traditional approaches to inventory tracking—whether

manual logbooks or rigid, expensive enterprise platforms—often lack the flexibility, accessibility, and real-time capabilities required by today’s agile business models. Delays in stock reconciliation, lack of centralized access, limited control over user roles, and complex workflows are all symptoms of these legacy systems.

Consider, for example, a mid-sized retail chain that maintains separate inventory records at each location using Excel sheets. Such a fragmented system leads to inconsistencies in stock levels, repeated orders of low-priority items, and the inability to track sales trends in real time. Similarly, small warehouse-based businesses often face difficulty reconciling physical stock with digital records due to delayed manual entries and inadequate user access management.

Stock-er bridges these gaps with a structured, scalable, and cloud-powered solution that combines intuitive design with powerful functionality. By integrating essential features like barcode-based item identification, real-time updates, and a multi-tier user role system, it enables businesses to manage their inventory dynamically without the steep learning curve or operational overhead commonly associated with traditional inventory software.

The platform’s development was also influenced by real user feedback and observations from small-to-medium businesses that reported challenges with usability, cost of ownership, and limited support in their current systems. The insights gathered helped shape Stock-er’s core design priorities—simplicity, security, and seamless collaboration across teams.

Security and usability are central to Stock-er's design. The platform supports secure user authentication mechanisms, structured user hierarchies (Admins, Managers, Staff), and fine-grained access control to ensure data integrity and accountability. The system’s built-in billing functionality and barcode support also help streamline day-to-day retail and warehouse operations, providing businesses with tools to operate more efficiently and accurately.

In addition, Stock-er's support for multi-tenant architecture ensures that each organization operates within its own isolated data environment—enabling distributed teams to collaborate while maintaining strict data separation. This makes the platform suitable for organizations with multiple branches or geographically dispersed teams.

As businesses continue moving toward cloud-native, collaborative ecosystems, Stock-er stands as a purpose-built, adaptable inventory management solution. Its focus on automation, secure data handling, and real-time collaboration positions it as a valuable tool for modern organizations seeking to overcome the limitations of legacy inventory management practices.

1.4 Objectives

The primary objective of the Stock-er Inventory Management System is to provide businesses—regardless of size or sector—with a smart, structured, and scalable solution for tracking inventory, managing stock flows, and enhancing operational efficiency. Stock-er has been developed with modern business needs in mind, addressing gaps in traditional inventory management systems through automation, usability, and cloud-based architecture.

Key Objectives

- **Optimize Inventory Tracking**

Offer real-time visibility into inventory levels, minimizing the risk of stockouts, overstocking, and discrepancies. Stock-er ensures that inventory data remains accurate, consistent, and up to date through automated synchronization across devices.

- **Enhance User Role Management**

Implement a multi-tier access system that defines clear responsibilities and access levels for Admins, Managers, and Staff. This ensures operational transparency, secure delegation, and accountability within the organization.

- **Improve Stock Categorization**

Allow businesses to organize and manage their inventory in logical, user-defined categories. This improves the clarity of inventory structure and enables faster search, retrieval, and reporting of stock data.

- **Integrate Barcode Processing**

Support the generation and scanning of standard barcode formats such as Code128, EAN-13, UPC-A, and more. Barcode support simplifies item tracking, speeds up billing and checkouts, and reduces human error.

- **Enable Secure Multi-Tenant Support**

Architect the system to function in a multi-tenant environment, ensuring that each organization's data is isolated, secure, and inaccessible to unauthorized users from other organizations.

- **Implement Efficient Billing and Checkout**

Provide a built-in billing module that allows users to apply flat or percentage-based discounts, calculate taxes, and generate professional receipts that can be saved and printed at a later stage.

- **Ensure Cloud-Based Accessibility**

Allow users to access their inventory data from any device, across any location, without data loss or mismatch. The system's cloud deployment guarantees consistent performance and real-time updates.

- **Strengthen Data Security and Authentication**

Protect sensitive data using encrypted credentials, secure access controls, and verification mechanisms such as email-based code verification for user onboarding and password recovery.

- **Facilitate Real-Time Synchronization**

Use event-driven technologies to enable real-time stock updates across users and devices, ensuring consistent inventory state and enabling collaborative workflows without lag or mismatch.

- **Reduce Operational Overhead**

Automate routine inventory tasks such as stock updates, categorization, and billing to reduce manual intervention, eliminate redundancy, and allow businesses to focus on higher-priority tasks.

In essence, Stock-er aims to deliver an intelligent inventory management platform that not only meets current operational requirements but also offers a flexible foundation for future enhancements—ultimately improving how businesses interact with, manage, and secure their inventory.

1.5 Purpose, Scope and Applicability

Purpose

The development of **Stock-er** is driven by the need to modernize and simplify inventory management across a broad spectrum of business operations. As organizations grow, the complexity of managing inventory—stock levels, item categorization, billing, and secure user access—can lead to inefficiencies and errors, particularly in systems that still rely on outdated or partially digital tools.

Stock-er addresses these challenges by offering a structured, scalable, and user-friendly platform that enables businesses to track inventory in real time, manage user roles effectively, and automate operational workflows. The system is designed to integrate seamlessly into different business models while minimizing implementation overhead and maximizing usability.

By digitizing inventory tracking and offering centralized cloud-based access, Stock-er helps organizations reduce manual workload, enhance accuracy, and make informed decisions based on real-time data. The platform supports multi-tenancy to ensure that each organization's data is securely isolated and privately managed, allowing for independent operations within a shared system environment.

Additionally, Stock-er serves as a professional-grade project that showcases full-stack software development capabilities. It reflects expertise in frontend/backend integration, database design, authentication protocols, cloud infrastructure management, and secure deployment practices. Through features like barcode processing, real-time synchronization, and role-based access control, Stock-er exemplifies practical problem-solving aligned with modern business expectations.

Key Purposes

- **Inventory Optimization:** Streamline stock tracking with structured workflows, reducing inconsistencies and improving operational control.
- **Multi-Tier Access Control:** Facilitate secure user management through hierarchical roles—Admins, Managers, and Staff—ensuring controlled operations.
- **Barcode-Based Stock Management:** Enhance accuracy and speed by assigning scannable, unique barcodes to inventory items.
- **Real-Time Inventory Updates:** Synchronize stock changes across all users and devices instantly to avoid conflicts or outdated records.
- **Secure Multi-Tenant Architecture:** Maintain strict data isolation between organizations, safeguarding privacy, and operational independence.
- **Integrated Billing & Checkout Module:** Support automated invoice generation with discount handling, tax calculation, and printable receipts.
- **Cloud-Based Accessibility:** Allow remote access and inventory management from any location, ensuring flexibility for modern businesses.
- **Security & Authentication:** Implement encrypted login, code-based user verification, and role-specific access for secure inventory handling.

- **Scalability:** Adapt to the needs of retail stores, warehouses, distributors, and corporate teams, regardless of business size or complexity.

Scope

Stock-er is a full-featured Inventory Management System (IMS) covering all core requirements for modern inventory handling. It is designed to bridge the gap between manual inventory tracking methods and complex enterprise systems, offering an efficient and easy-to-use alternate that is particularly well-suited to small- and medium-sized enterprises (SMEs) looking to digitize their workflows.

- **Inventory Tracking & Categorization:** Maintain records of stock items, including descriptions, pricing, and quantity within categorized groupings.
- **Barcode Generation & Integration:** Generate and scan item-specific barcodes (supporting the formats Code11, Code39, Code93, Code128, UPC-A, UPC-E, EAN-8, EAN-13, ITF-14 and Interleaved 2 of 5) to speed up transactions and simplify identification.
- **Multi-Tenant System Support:** Handle data securely for multiple organizations in a shared environment while maintaining strict separation of records.
- **Role-Based User Management:** Define, manage, and enforce role-based access policies across Admin, Manager, and Staff tiers.
- **Billing & Checkout Processing:** Automate transaction workflows with options for applying discounts, calculating taxes, and generating receipts that are stored and can be printed later.
- **Real-Time Data Synchronization:** Reflect changes across users and devices instantly, minimizing delay and preventing data mismatches.

- **Cloud-Based Deployment:** Provide continuous, reliable access through a cloud-hosted backend, ensuring operational continuity and scalability.
- **Security & Authentication Features:** Protect user sessions and sensitive data through encrypted communication, email verification, and secure password handling during login, signup, or recovery workflows.

While Stock-er currently focuses on operational essentials, its architecture allows for future expansion into areas like reporting dashboards, predictive analytics, and supply chain integration.

Applicability

Stock-er is suited for a wide range of use cases across multiple industries, thanks to its modular design and flexible feature set. Whether used in retail, wholesale, logistics, or office administration, Stock-er brings structure, visibility, and security to the inventory management process.

Examples of applicable scenarios include:

- **Retail Businesses** managing point-of-sale transactions, shelf stock, and billing.
- **Wholesale Distributors** tracking high-volume goods movement and barcode-tagged inventory.
- **Warehouse Teams** requiring collaboration and live updates on stock positions.
- **Corporate Offices** managing equipment, consumables, and inter-departmental supply chains.
- **E-commerce Platforms** needing inventory tracking, secure billing, and role-specific operations for fulfilment teams.

By delivering a modern, responsive, and scalable IMS, Stock-er empowers businesses to move beyond the limitations of legacy systems—bringing operational agility, data integrity, and user accountability into focus.

Chapter 2: Survey of Technology

2.1 Technologies

Stock-er is built upon a modern, flexible technology stack designed to support scalability, maintainability, and ease of development. The tools and platforms used span across integrated development environments, database systems, programming frameworks, and version control utilities, collectively supporting a full-stack development approach.

Development & IDE Tools

- **Visual Studio:** The primary Integrated Development Environment (IDE) used for coding, debugging, and managing Stock-er's application logic using C# and the .NET framework.

Programming Language & Framework

- **C#:** The main programming language used to build the system's core logic, user role management, billing workflows, and backend services.
- **.NET Framework:** A powerful and scalable framework used to build backend functionalities, handle authentication, and support integration with databases and other services.

Database & Data Management

- **MongoDB:** A NoSQL document database that stores all inventory records, user roles, billing data, and transaction logs in a flexible, schema-less format, allowing for efficient querying and updates.
- **MongoDB Compass:** A graphical user interface tool that enables developers to inspect and interact with MongoDB data during the development and debugging process.

- **MongoDB Atlas (deployed via Azure):** A cloud-hosted database service that provides a scalable, secure, and reliable production database environment. Atlas ensures high availability and enables real-time synchronization across distributed users and locations.

Version Control & Collaboration

- **Git:** A distributed version control system used to track source code changes, facilitate branching and merging, and ensure team collaboration.
- **GitHub:** A web-based Git repository hosting platform used for code sharing, issue tracking, documentation, and collaboration throughout the development lifecycle. GitHub also supports version control transparency and rollback capabilities during updates or modifications.

Package Management

- **NuGet:** The official package manager for the .NET ecosystem used to install, update, and manage third-party libraries and tools. NuGet ensures that all project dependencies are consistently maintained and easily restorable, improving build reproducibility and reducing setup overhead for new developers.

These technologies collectively enable Stock-er to provide a reliable, cloud-ready inventory management system while ensuring high standards of code organization, data security, and real-time access across distributed environments.

2.2 Frontend Modules

The frontend of the Stock-er Inventory Management System is developed using **WinForms** with enhanced visuals and components provided by third-party UI libraries. The frontend ensures an intuitive, modern, and user-friendly interface for all system roles (Admins, Managers, Staff), supporting structured workflows and seamless interaction.

The key packages used in the frontend include:

- **Guna.UI2.WinForms:**

Provides modern, stylized WinForms UI components such as buttons, panels, progress indicators, and data grids. This enhances the application's visual appeal and improves user experience compared to traditional WinForms controls.

- **BarcodeLib:**

Enables barcode generation functionality directly from the frontend. It supports creating standard 1D barcodes for labeling and managing inventory items efficiently.

- **SkiaSharp:**

A cross-platform 2D graphics library used for rendering high-quality graphics and improving visual outputs like dynamically generated charts or barcode rendering.

- **iTextSharp:**

Integrated for generating and exporting PDF documents such as invoices and billing receipts directly from the frontend interface.

2.3 Backend Modules

The backend handles data processing, database communication, authentication, and synchronization. Developed in C# using the **.NET Framework**, it supports robust multi-tenant architecture, real-time updates, and secure stock management operations.

Key backend libraries and packages include:

- **MongoDB.Driver & MongoDB.Bson:**

Used to connect and interact with MongoDB, handling document-based data operations including CRUD (Create, Read, Update, Delete) for inventory items, users, organizations, and transactions.

- **Newtonsoft.Json:**

A widely used library for JSON serialization and deserialization, allowing smooth communication between application components and storage in structured formats.

- **BCrypt.Net-Next:**

Provides secure password hashing for user credentials using the bcrypt algorithm, enhancing authentication security.

- **ZXing.Net:**

Handles barcode and QR code scanning. It complements the barcode generation feature by allowing users to decode printed labels during stock updates or checkout processes.

- **Microsoft.Extensions.Hosting:**

Supports background service hosting, making it easier to manage long-running tasks, event-driven updates, and system services efficiently.

Chapter 3: Requirement Analysis

3.1 Problem Definition

Inventory management plays a crucial role in ensuring smooth operations across various industries, including retail, wholesale, warehousing, and supply chain logistics. However, many businesses continue to struggle with outdated or fragmented systems that lack integration, real-time visibility, and user-friendly interfaces. These limitations often lead to operational inefficiencies such as inaccurate stock levels, delayed restocking, data inconsistencies, and human errors.

Traditional inventory tracking methods—whether through manual logbooks, spreadsheets, or legacy software—fail to scale effectively with growing organizational needs. They offer limited multi-user support, lack secure access controls, and often provide minimal automation. As a result, organizations face challenges such as:

- Inability to monitor stock levels in real time, leading to overstocking or stockouts.
- Lack of structured user roles, making accountability and access control difficult.
- Absence of automation for repetitive tasks like barcode scanning, billing, and report generation.
- Difficulty in managing data securely across multiple teams or departments.
- Poor system usability, leading to increased training overhead and user errors.

Moreover, most existing inventory systems come with high setup complexity, lack of cloud-based accessibility, and limited adaptability for multi-tenant environments. These issues present significant bottlenecks, especially for small-to-medium businesses that require streamlined yet robust solutions.

Stock-er is developed in response to these challenges. It aims to deliver a secure, real-time, and intuitive inventory management system tailored to modern business needs. By incorporating features like barcode integration, role-based access, multi-tenant data isolation, and real-time synchronization, Stock-er addresses the inefficiencies of conventional solutions and provides businesses with a reliable, scalable, and user-centric platform for managing inventory.

3.2 Requirement Specification

The development of the Stock-er Inventory Management System is driven by the need to address a range of operational challenges commonly faced by businesses when managing stock. These requirements are categorized into **functional** and **non-functional** specifications to ensure a comprehensive foundation for implementation and evaluation.

3.2.1 Functional Requirements

Functional requirements define the specific capabilities and features that Stock-er must deliver to meet user and business expectations.

1. User Authentication and Account Management

- The system must allow users to register using an invite code, upon which a verification code is emailed to complete the signup process.
- Login is secured via single-factor authentication, utilizing encrypted credentials.
- Users who forget their password must be able to reset it through a secure verification code sent to their registered email.
- Account roles must be assignable and editable only by Admin-level users.

2. Role-Based Access Control

- Stock-er supports a hierarchical user role system, consisting of Admin, Manager, and Staff roles.
 - **Admin:** Has full access to organizational settings, billing modules, user management, and inventory configuration.
 - **Manager:** Has authority over inventory operations and staff-level management.
 - **Staff:** Restricted to viewing and modifying inventory items under supervised roles.
- Role-based access determines what modules, data, and actions are available to the user based on their designation.

3. Multi-Tenant Architecture

- The system must isolate data per organization to maintain privacy and data integrity across tenants.
- Each organization operates independently within the system, including user roles, inventory items, billing records, and configurations.
- No user from one organization should have visibility or access to another organization's data, regardless of user role.

4. Inventory Item Management

- Users must be able to add, update, and delete inventory items, including fields such as item name, description, quantity, unit price, and category.
- Items should be categorized into user-defined groups for structured management and efficient filtering.
- The system must support the attachment of **up to 10 images per item**, enabling visual identification and enhanced item documentation.

5. Barcode and QR Code Integration

- Each item must have the ability to generate a unique barcode and/or QR code.
- Users must be able to scan barcodes or QR codes to retrieve or update item information instantly.
- Barcode scanning must support both USB scanner hardware and keyboard input emulation for compatibility across devices.

6. Billing and Checkout Module

- The system should support the creation of itemized bills directly from the inventory.
- Users must be able to apply discounts either as flat values or percentages.
- Tax calculation must be automated based on predefined tax rates.
- Each bill must be stored in the system, linked to the respective user and organization.
- Bills must be downloadable and printable in PDF format, with the option to reprint at any time.

7. Real-Time Synchronization

- All inventory changes, whether item updates or billing actions, must reflect instantly across all logged-in user devices.
- The system should use event-driven data synchronization to ensure consistency without manual refreshing.

8. Search and Filtering

- The system must offer full-text search across item names, categories, and barcodes.
- Filtering options must be available based on category, stock level thresholds, or date added.

9. Reporting and Logs

- Users must be able to generate reports on inventory status, billing history, and user activity.
- Logs should capture item changes, stock updates, and billing transactions for audit purposes.

3.2.2 Non-Functional Requirements

Non-functional requirements describe the system's quality attributes and operational conditions.

1. Usability

- Stock-er must have an intuitive and visually clean user interface built with Guna.UI2.WinForms.
- Layout should be optimized for productivity, with minimal learning curve for new users.
- Clear labels, responsive controls, and feedback prompts are essential to guide user interaction.

2. Performance

- The system must handle large inventories (thousands of items) without latency in search, sync, or load operations.
- Data operations must complete within acceptable thresholds (e.g., inventory search results under 2 seconds).

3. Reliability and Availability

- Stock-er must maintain high availability, ensuring the system is accessible across time zones and working hours.
- Use of MongoDB replica sets ensures data redundancy and disaster recovery options.

4. Security

- All data transmissions must be encrypted using SSL/TLS protocols.
- Passwords are hashed using strong one-way encryption (BCrypt).
- Role-based access limits sensitive actions to authorized users.
- The system must enforce organizational data isolation to prevent leaks between tenants.

5. Maintainability

- Code should follow modular architecture, allowing easy updates to individual components without affecting the whole system.
- Git version control must be used for all development changes, with consistent commit history and branching strategies.

6. Scalability

- The system must be scalable to accommodate growing businesses with expanding inventory and user bases.
- Cloud-based architecture support horizontal scaling of user sessions and data handling.

7. Portability and Compatibility

- Stock-er must be compatible with standard Windows environments.
- It should support barcode scanners that operate as USB input devices.

8. Data Integrity

- The system must validate all input fields and prevent malformed or incomplete entries.
- All changes to critical data should be logged, and backups should be performed regularly.

3.3 SDLC Models

The development of the **Stock-er Inventory Management System** followed an **Iterative Software Development Life Cycle (SDLC) Model**, which allowed the system to evolve through repeated refinement across development cycles.

This model was selected for its flexibility in accommodating incremental feature rollouts, continuous feedback integration, and progressive enhancement without compromising structural integrity.

3.3.1 Why the Iterative Model?

Unlike linear models where all requirements must be finalized upfront, the Iterative model supports the progressive construction of the system. Stock-er, as a real-world inventory management solution, required this adaptability due to:

- **Incremental Feature Development:** Core modules such as inventory tracking, billing, barcode handling, and multi-user roles were developed and refined one at a time.
- **Feedback Integration:** Adjustments were made throughout the development cycle based on hands-on usage, testing feedback, and evolving goals.
- **Risk Mitigation:** Early development cycles helped identify issues in logic flow, data synchronization, and usability, reducing major risks in later stages.
- **Scalable Expansion:** New functionalities like image attachments and real-time sync were seamlessly integrated in later cycles without overhauling earlier code.

3.3.2 Iterative Phases

Each development cycle included a complete pass through the essential SDLC phases for a particular set of features:

1. Planning and Analysis

Before each iteration, requirements were outlined based on feature goals—whether it was setting up multi-tenant support, implementing barcode scanning, or enhancing role-based access.

2. Design

Modular designs were drafted to handle each feature independently. Reusable UI components and backend routines were prioritized to maintain consistency.

3. Implementation

Functionality was coded using C# in the .NET Framework environment. MongoDB was integrated as the backend for its flexibility and scalability. Each iteration included Git-based version control and branching for safe rollouts.

4. Testing

Post-development, each feature set underwent testing—ranging from input validation and authentication checks to real-time inventory synchronization. Bug fixes and optimizations followed quickly within the same cycle.

5. Review and Enhancement

After validation, the implemented features were reviewed. Based on observed behavior or user-side testing, enhancements and refinements were planned into the next iteration.

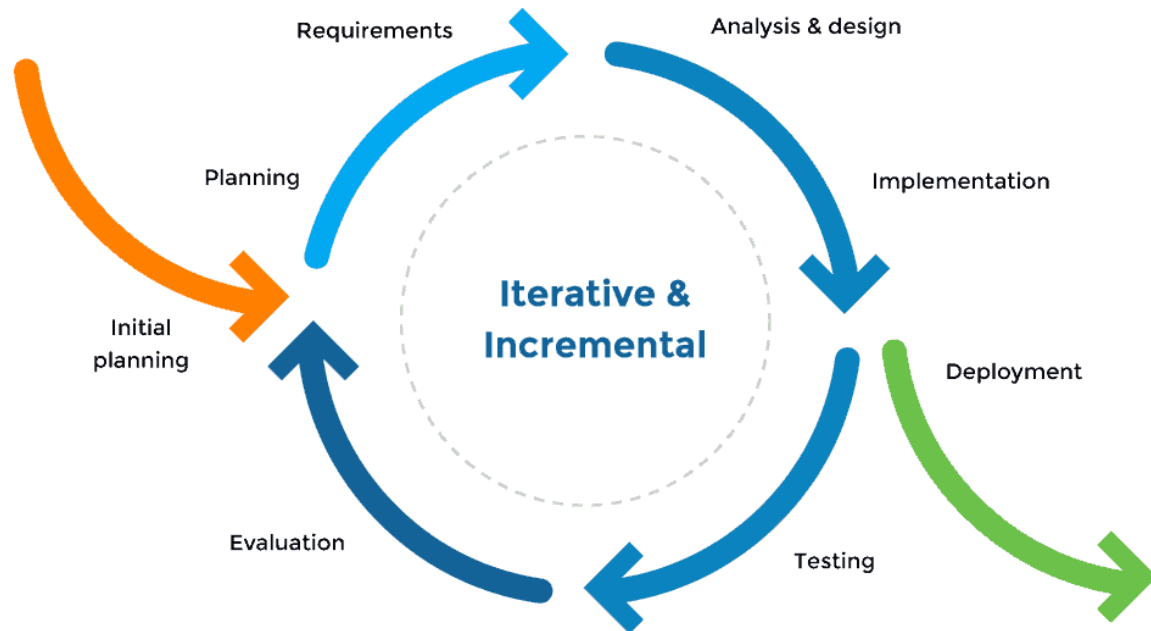


Figure 3.3.2 – Iterative Model

3.3.3 Advantages of This Approach

Adopting the **Iterative SDLC Model** for the development of *Stock-er* offers several advantages that align with the nature of the system and its evolving requirements:

- **Progressive Feature Development:** The system is developed in iterations, allowing early core functionalities—such as inventory tracking and role-based access—to be operational while other components are still under construction.
- **Early Detection of Issues:** Since each cycle includes development and testing, bugs and design flaws can be identified and addressed early, reducing the likelihood of large-scale system failures later.
- **Better Stakeholder Involvement:** Stakeholders can review and provide feedback on functioning modules early in the process. This fosters transparency and ensures the final product aligns with user expectations.

- **Improved Flexibility:** Changing requirements—be it in user roles, workflows, or inventory processes—can be more easily integrated into future iterations without disrupting the existing system.
- **Risk Mitigation:** Smaller development cycles make it easier to isolate and manage risks. If an issue arises, it's often limited to a specific iteration and can be resolved without impacting the entire project.
- **Parallel Development Possibility:** Different development teams can work simultaneously on separate modules or upcoming iterations, expediting the build process while maintaining modularity.
- **Better Resource Allocation:** Effort, time, and resources can be distributed more efficiently across iterations based on priority and complexity, ensuring optimal productivity.
- **Continuous Refinement:** The ability to improve and optimize features over successive iterations leads to a more polished and user-friendly final product.

This approach ensures that *Stock-er* evolves alongside user feedback and operational realities, making it more robust, scalable, and adaptable than systems built under rigid linear models.

3.4 Planning and Scheduling

The development of *Stock-er* followed a structured and time-bound plan using iterative development principles, where each cycle focused on designing, implementing, and refining key modules.

The planning process was driven by both functional priorities and technological dependencies, ensuring the system evolved incrementally while allowing continuous integration and testing.

3.4.1 Development Phases

The entire development process was broadly divided into the following phases:

1. Requirement Gathering and Analysis

- Identification of inventory challenges in existing systems
- Gathering user expectations and defining scope
- Documenting system objectives and functional needs

2. System Design

- Planning of multi-tenant architecture and user role hierarchy
- Designing entity relationships, workflows, and data schemas
- Creating UML diagrams to visualize interactions and structure

3. Module-wise Implementation

- Inventory core (add, update, delete, search)
- Barcode and QR code integration
- Billing, discounting, and receipt generation
- Authentication and role-based access control
- Real-time synchronization mechanisms

4. Testing and Optimization

- Iterative testing of functional units and UI components
- Data integrity validation and exception handling

- Usability adjustments and bug fixes

5. Deployment and Documentation

- Hosting with cloud-based database (MongoDB Atlas)
- Generating project documentation and user manuals
- Preparing codebase for portfolio submission

3.4.2 Timeline Overview

A typical iteration cycle lasted approximately **2–3 weeks**, depending on module complexity. Below is a high-level timeline:

Week(s)	Phase	Deliverables
<i>Week 1–2</i>	Requirement Analysis + Planning	Requirement doc, initial tech stack design
<i>Week 3–5</i>	System Design + Architecture	ER diagrams, Use Case, DFDs, UI wireframes
<i>Week 6–8</i>	Inventory & Barcode Module	Inventory CRUD, barcode generation/viewing
<i>Week 9–10</i>	Billing System	Discount handling, tax logic, PDF generation

Week 11–12	Authentication + Role Management	Secure login, invite/join via email, role UI
Week 13–14	Real-Time Sync	Device sync, data conflict handling
Week 15	Testing & Debugging	Bug fixing, edge case testing
Week 16	Final Documentation + Deployment	User docs, report writing, system release

Table 3.4.1.1 - Timeline Overview

This structured planning approach enabled progressive development with continuous integration of modules, timely identification of blockers, and ensured that the final build of *Stock-er* met both functional and technical expectations.

3.4.3 Gantt Chart

ID	Task Name
1	▼ Project Duration
2	Requirement Analysis and Planning
3	System Design & Architecture
4	Inventory Module
5	Barcode Module
6	Billing System
7	Authorization and Role Management
8	Real-Time Sync Implementation
9	Testing and Debugging
10	Final Documentation and Deployment

Figure 3.4.3.1 – Tasks List

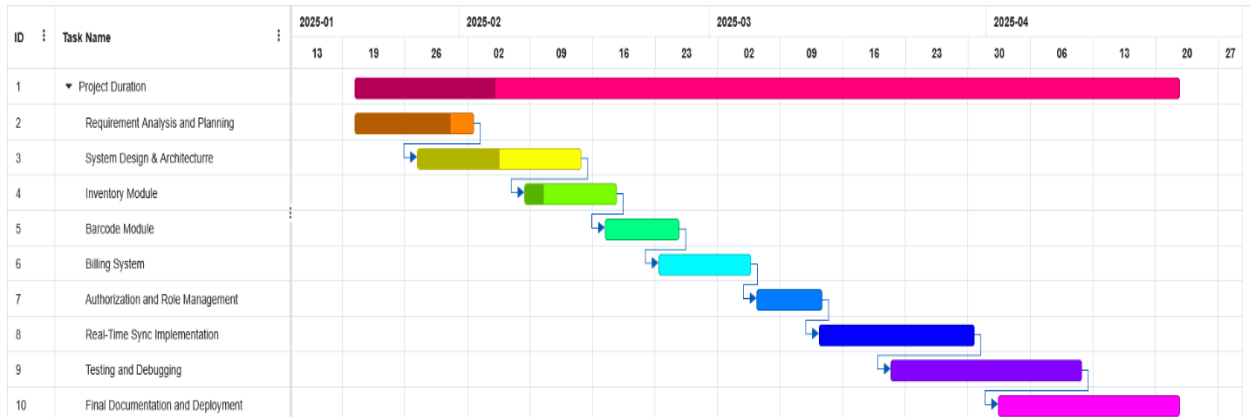


Figure 3.4.3.2 – Gantt Chart

3.5 Software and Hardware Requirements

For the successful deployment and operation of the Stock-er Inventory Management System, a set of minimum software and hardware requirements must be met. These requirements are categorized based on the development environment and the end-user environment and account for the necessary infrastructure to support a cloud-based architecture.

The Stock-er system is designed with flexibility in mind—supporting lightweight local setups while leveraging a robust cloud backend through MongoDB Atlas. With built-in USB HID scanner compatibility, remote database access, and a user-friendly Windows-based application model, Stock-er offers seamless functionality and secure, real-time data transactions. These requirements ensure the system remains suitable for both small businesses and scalable enterprise environments.

3.5.1 Development Environment Requirements

These are the tools and configurations used during the development and testing phases of the Stock-er system:

Software Requirements:

- **Visual Studio (2022 or later):** Primary Integrated Development Environment (IDE) used for designing, coding, debugging, and testing the C# .NET Windows Forms application.
- **.NET Framework 4.8 or higher:** Framework targeted by the Stock-er system, enabling compatibility with modern Windows environments and rich UI support.
- **MongoDB Atlas:** Cloud-based NoSQL database used during development and testing for simulating live connectivity and remote database operations.
- **MongoDB Compass:** GUI-based tool for interacting with MongoDB databases, performing queries, and monitoring document structures.
- **Git:** Version control system used to manage source code changes and maintain development history.
- **GitHub:** Remote repository hosting platform utilized for code collaboration, backup, and deployment pipeline management.

Hardware Requirements:

- **Processor:** Intel i5 (10th Gen or higher) or AMD Ryzen 5 equivalent
- **RAM:** Minimum 8 GB (16 GB recommended for smooth debugging and emulation)
- **Storage:** SSD with at least 10 GB free for project files, dependencies, and databases
- **Display:** 1080p screen resolution or higher for optimal Visual Studio layout
- **Peripherals:** USB support for barcode scanner testing (USB HID-compatible)

3.5.2 End-User System Requirements

These are the requirements for the machines on which the Stock-er system will be installed and operated by end users within an organization.

Software Requirements:

- **Windows OS (Windows 10 or higher):** The system is designed as a Windows desktop application using WinForms and is compatible with all modern versions of Windows.
- **.NET Framework 4.8:** Required runtime environment to launch and operate the Stock-er executable.
- **Internet Connectivity:** Essential for connecting to the cloud-hosted MongoDB Atlas instance. A stable internet connection ensures real-time synchronization of inventory data across all users.
- **MongoDB Atlas Connectivity:** As the backend database is hosted on MongoDB Atlas, users do not need to install any local database servers. Instead, data is fetched and updated securely over the internet.
- **Security Support:** SSL/TLS support enabled by default in modern Windows systems ensures that all cloud communication is encrypted.

Hardware Requirements:

- **Processor:** Dual-core 2.4 GHz or higher
- **RAM:** Minimum 4 GB (8 GB recommended for better responsiveness)
- **Storage:** 1 GB of free space to install and run the Stock-er application and store temporary data

- **Display:** 1366 x 768 minimum resolution; 1920 x 1080 recommended for optimal UI experience
- **Peripheral Compatibility:**
 - **USB HID Barcode Scanner:** The system supports barcode scanners functioning as USB Human Interface Devices (plug-and-play). These are used for scanning product barcodes during stock entry and checkout processes.
 - **Printer:** Required for bill/receipt generation. The system generates PDF receipts, which can be printed via any connected printer.

3.5.3 Infrastructure Requirements

Given that Stock-er supports multi-tenancy and real-time synchronization, certain backend infrastructure aspects are implicitly required:

- **MongoDB Atlas Cluster:** Cloud-based database instance with role-based access control, IP whitelisting, and real-time performance metrics.
- **Deployment Environment (Optional):** Organizations can host the Stock-er executable on an internal file server or distribution portal for easier deployment across teams.
- **Backup/Recovery Support:** MongoDB Atlas provides automated backups and snapshots, which are essential for disaster recovery and data protection.

3.6 Preliminary Product Description

The **Stock-er Inventory Management System** is a modern, desktop-based inventory control solution tailored for businesses requiring a structured, scalable, and efficient approach to stock management. Built on the .NET Framework with a C# backend, Stock-er integrates seamlessly with MongoDB Atlas to offer cloud-powered data persistence, enabling cross-device accessibility and remote database management.

Designed with both usability and functionality in mind, Stock-er features a role-based access system that categorizes users into Admin, Manager, and Staff roles—each with specific privileges and workflow responsibilities. This ensures that organizational operations remain well-defined, secure, and auditable.

The core of Stock-er revolves around its inventory tracking mechanism, allowing businesses to maintain detailed records for each item, including name, SKU, category, pricing, quantity, supplier info, and up to **10 image attachments** per item for visual verification and auditing. The system also supports real-time updates and device synchronization, ensuring consistent and current inventory data across all connected systems.

Barcode integration is one of the defining features of Stock-er. To streamline the stock retrieval and checkout process, the system provides **three flexible modes** of barcode usage:

- **Hardware Scanner Input** – Compatible with USB HID barcode scanners for quick and plug-and-play item scanning.
- **Manual Barcode Entry** – Users can directly type the barcode string into the system interface in case a scanner is unavailable or malfunctioning.
- **Barcode Image Upload** – The application supports uploading a barcode image from the device for decoding and item identification, covering edge cases such as remote assistance or digital barcode sharing.

In addition, the system includes a billing and checkout module that facilitates quick invoice generation with tax calculations and optional discount applications (flat or percentage-based), making it well-suited for customer-facing or internal transaction environments.

The application employs a modular structure, with clearly separated components for inventory handling, billing, authentication, barcode/QRCode utilities, and administrative controls. Security is reinforced with encrypted user authentication credentials and verification-based flows for critical operations such as sign-up, invite-based onboarding, and password resets.

Cloud architecture ensures that businesses can access and manage inventory from any location with internet connectivity, while MongoDB Atlas provides high availability, backup, and scalability. Stock-er's clean, responsive user interface—enhanced by Guna.UI2.WinForms components—ensures a user-friendly experience without overwhelming visual clutter.

In summary, Stock-er presents a fully integrated inventory management ecosystem that combines essential stock handling tools with modern cloud accessibility, intuitive design, and enterprise-level security to cater to the operational needs of growing businesses.

3.7 Conceptual Models

To effectively illustrate the system's design and operational logic, several conceptual models have been developed. These models serve as high-level visual representations of the system's structure, data flow, user interactions, and core functionalities. They help bridge the gap between abstract requirements and technical implementation, providing a clear understanding of how different components and actors interact within Stock-er. The models include diagrams such as the Use Case Diagram, Data Flow Diagram (DFD), and Class Diagram, each offering unique perspectives on the system's architecture and behaviour.

3.7.1 Activity Diagram

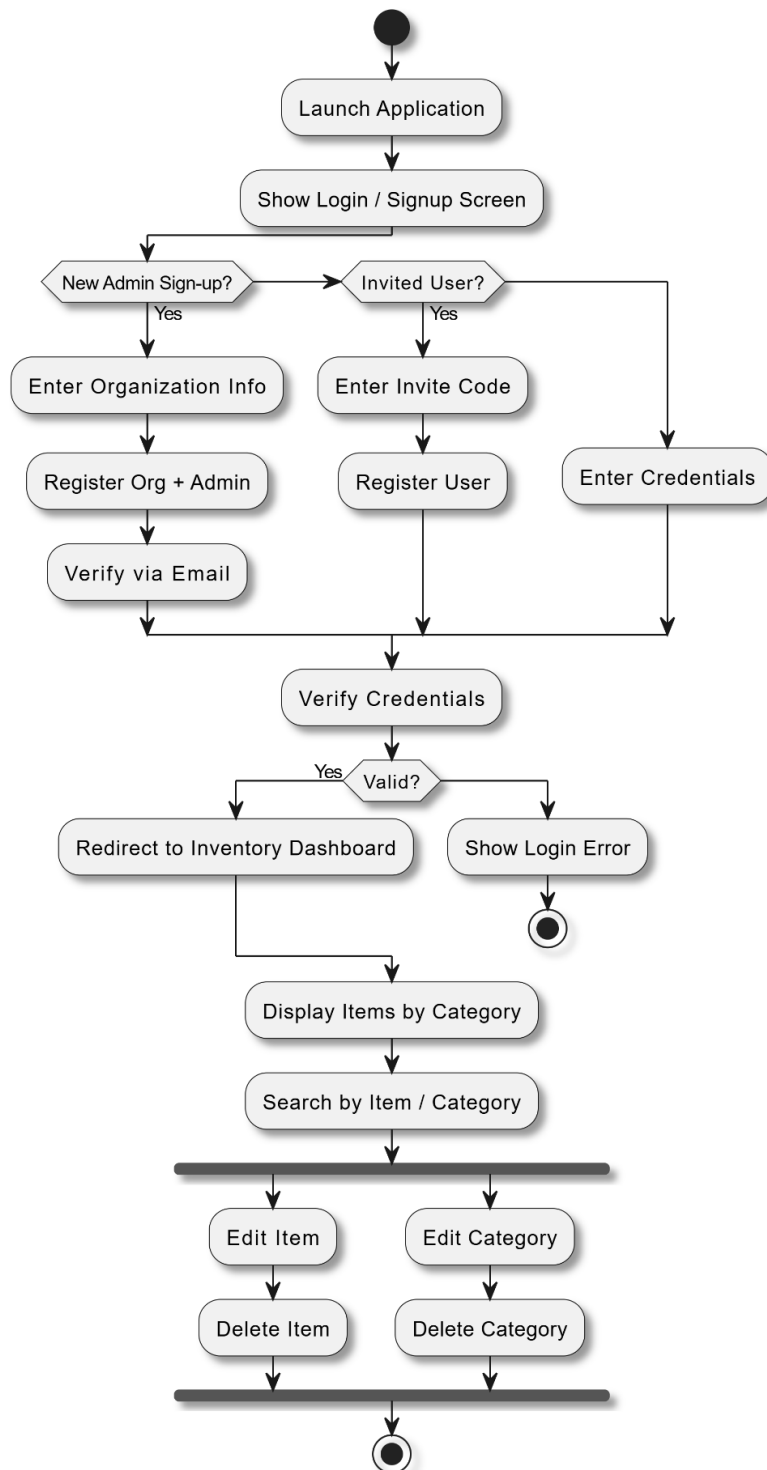


Figure 3.7.1.1 – Activity Diagram -1

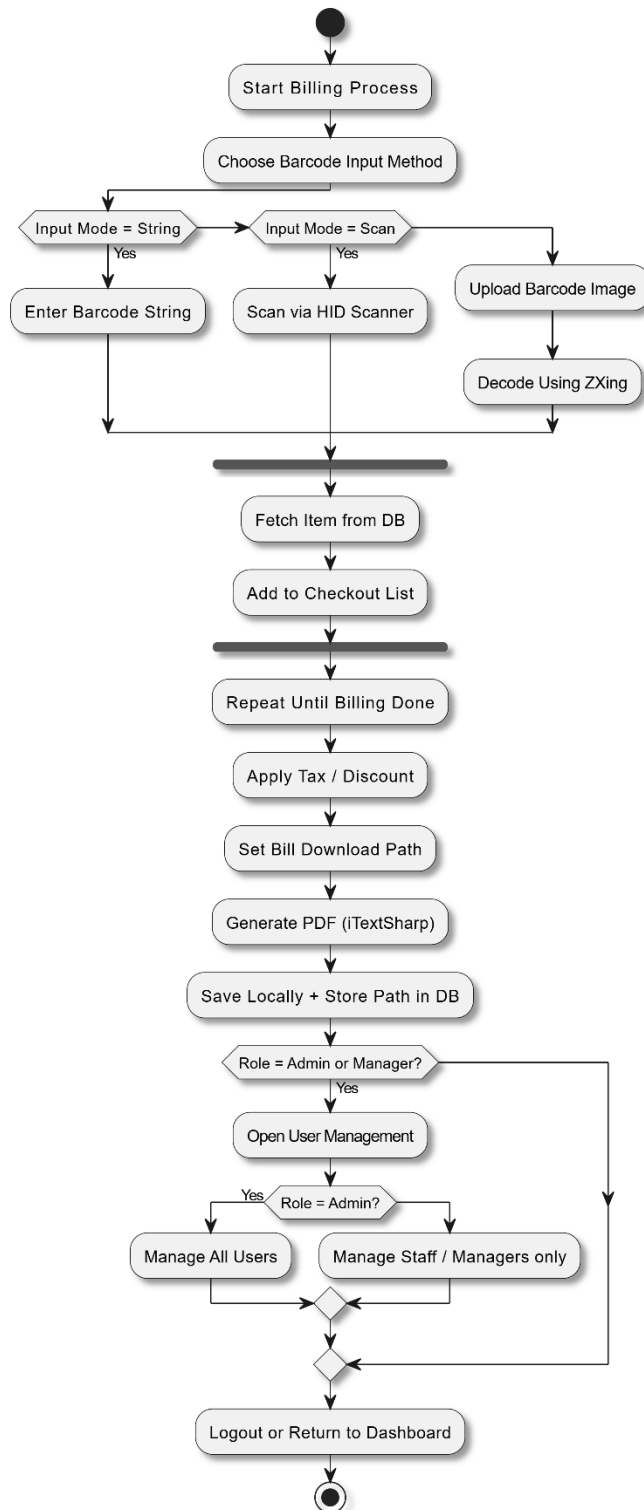


Figure 3.7.1.2 – Activity Diagram -2

3.7.2 Sequence Diagram

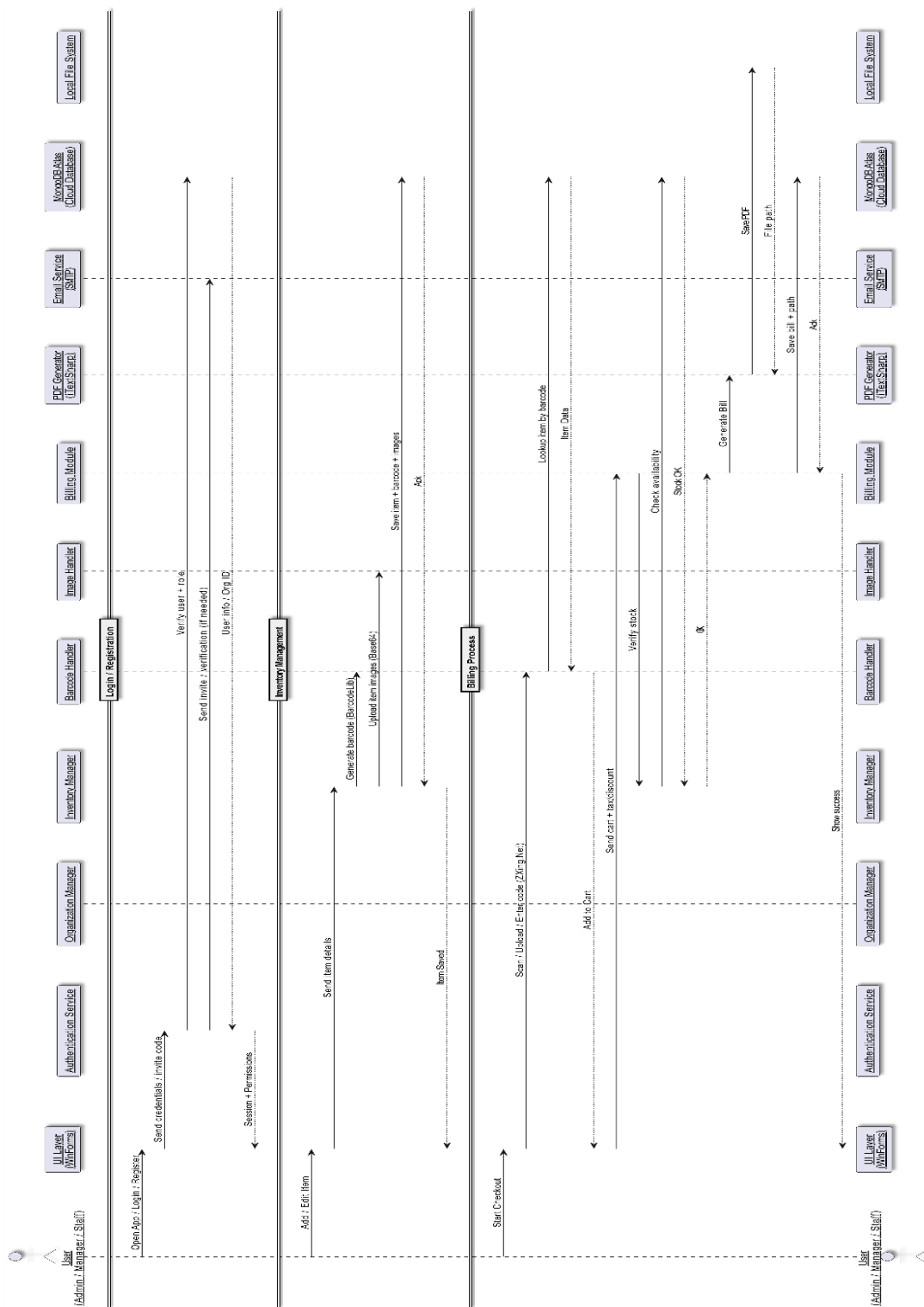


Figure 3.7.2.1 –Sequence Diagram

3.7.3 Entity-Relationship Diagram (ERD)

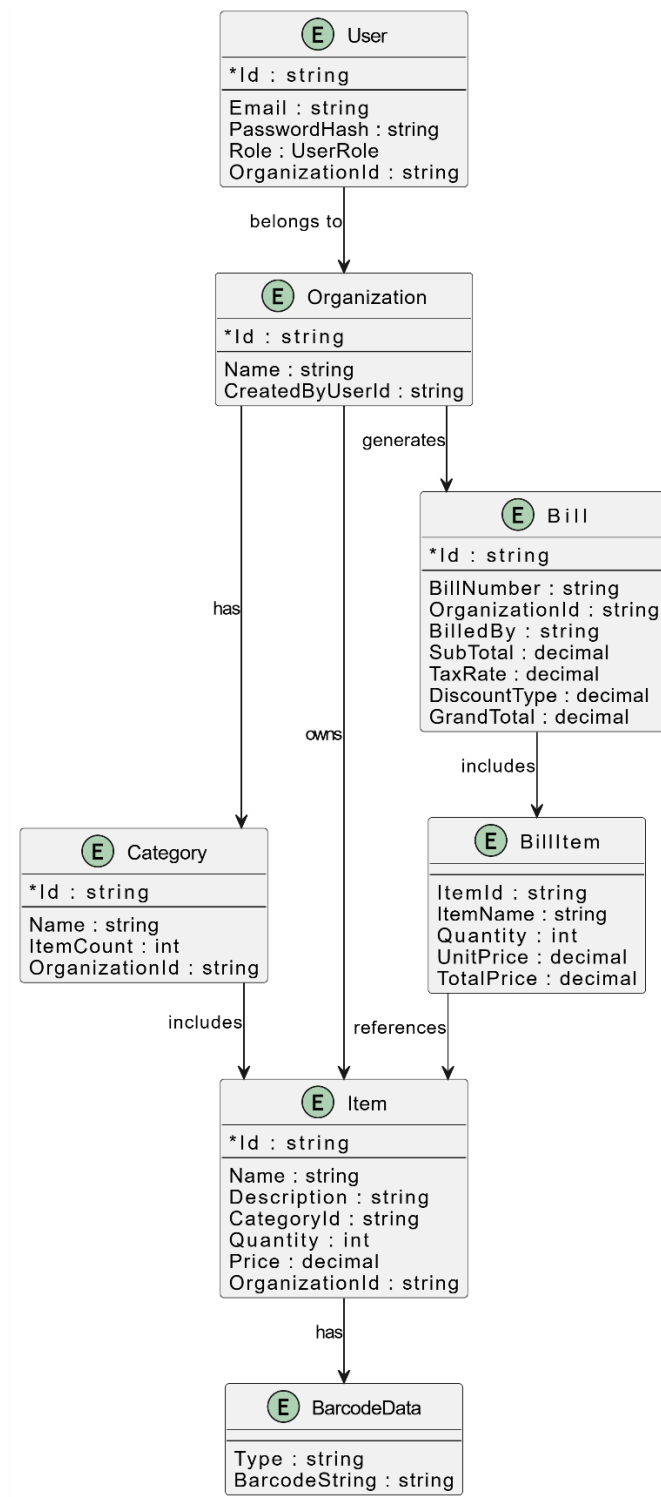


Figure 3.7.3.1 – Entity-Relationship (ER) Diagram

3.7.4 Use Case Diagram

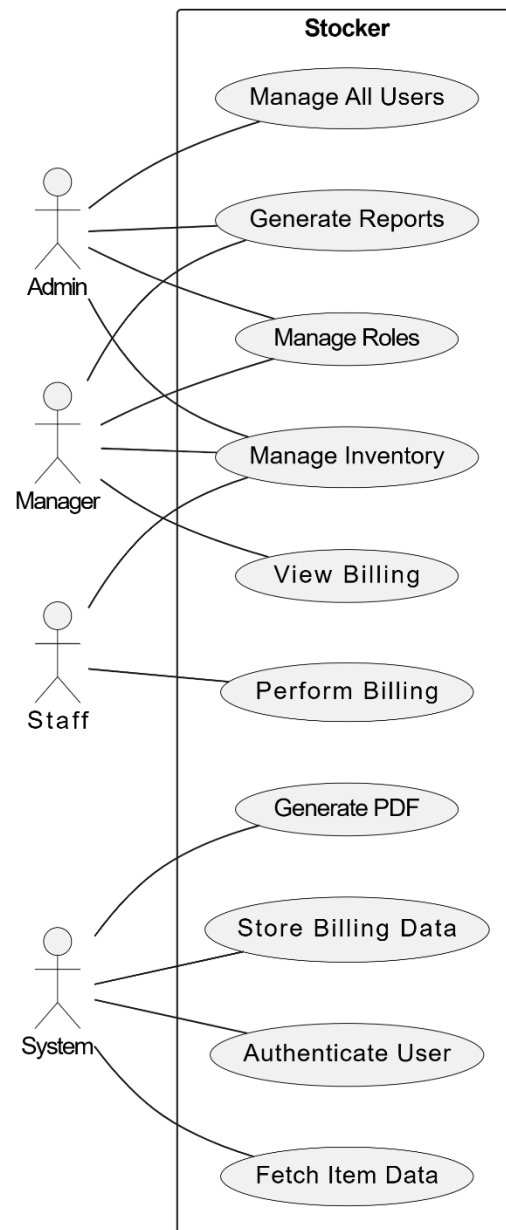


Figure 3.7.4.1 – Use Case Diagram

3.7.5 Data Flow Diagram (DFD)

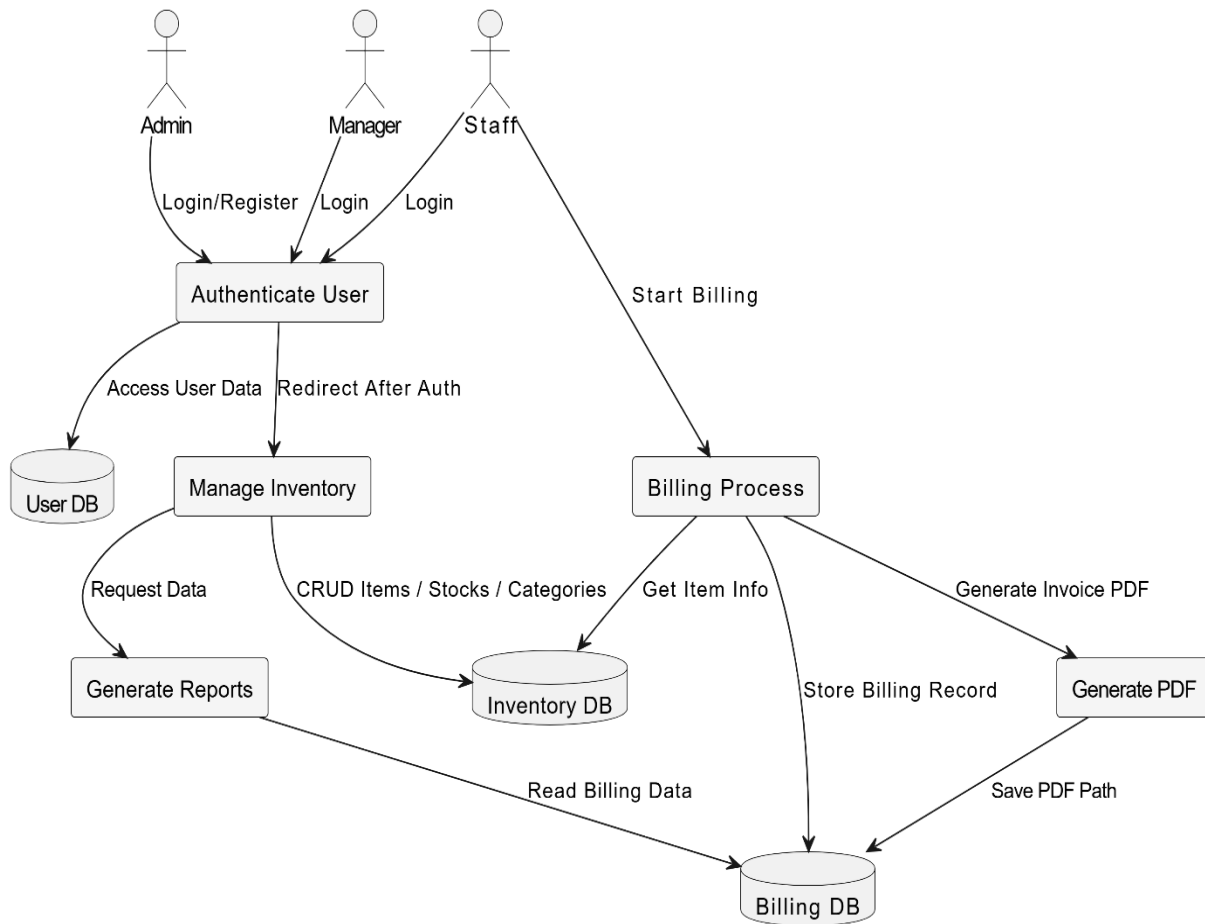


Figure 3.7.5.1 – Data Flow Diagram

3.7.6 Component Diagram

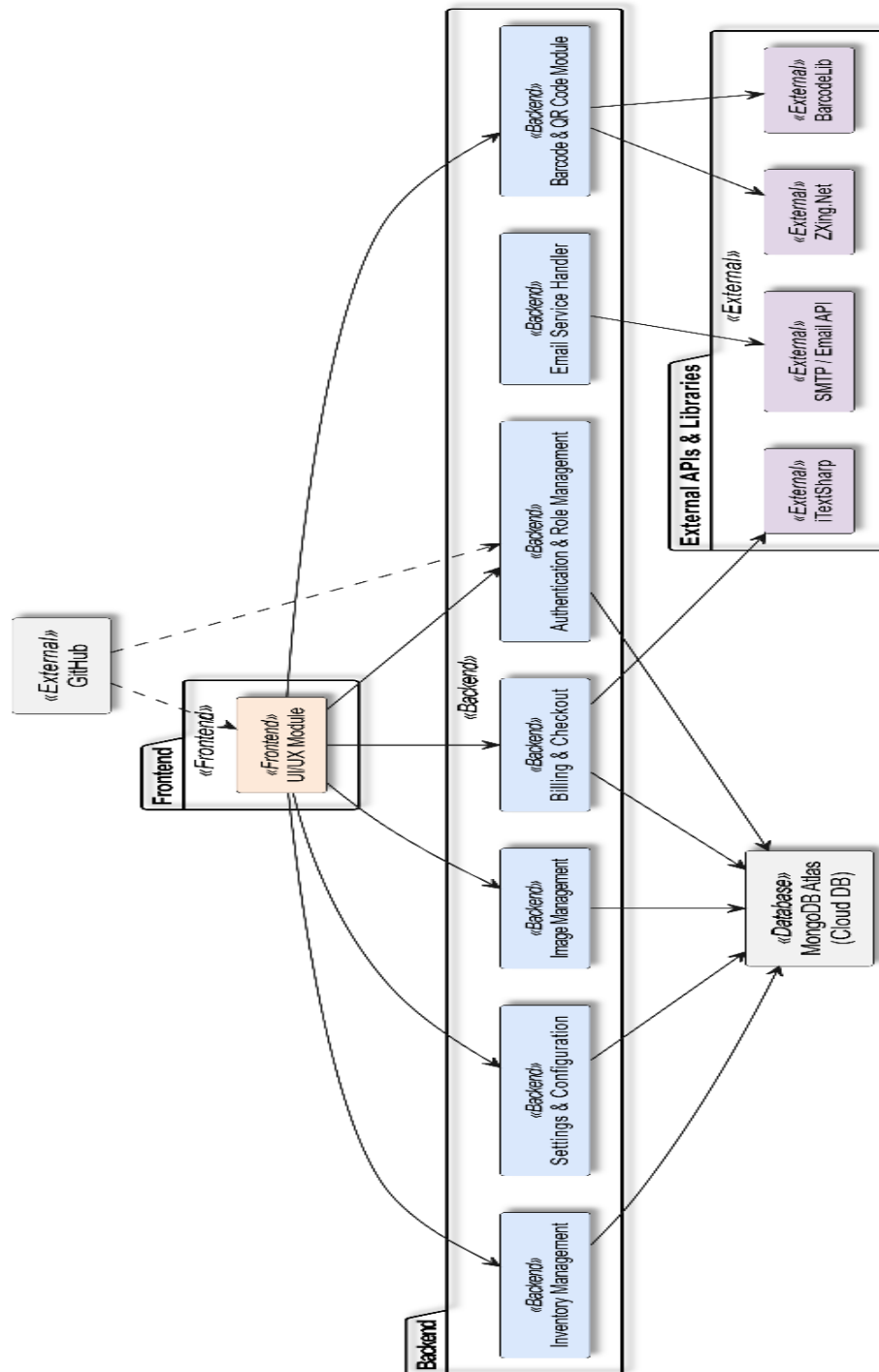


Figure 3.7.6.1 – Component Diagram

3.7.7 System Architecture Diagram

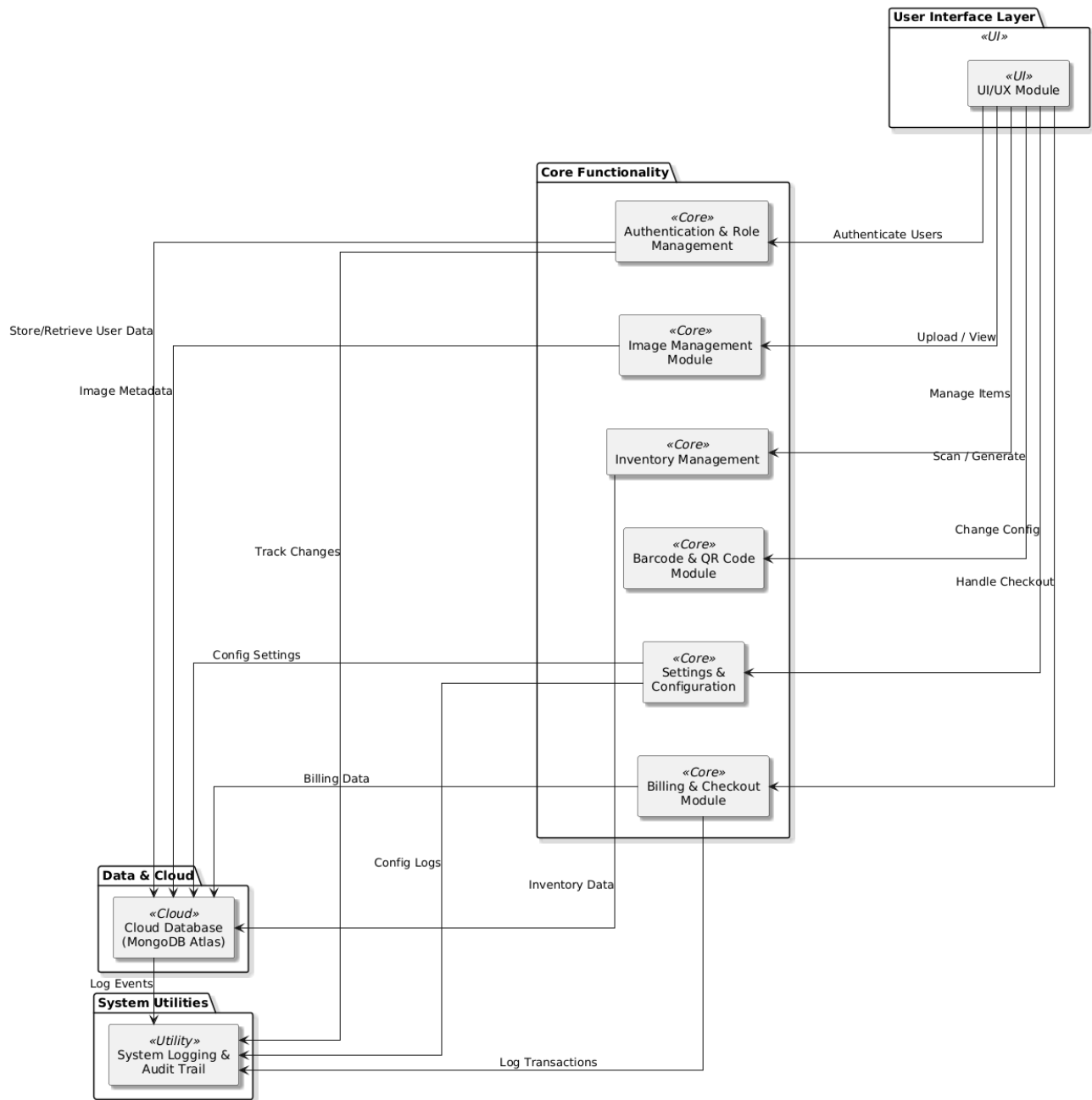


Figure 3.7.7.1 – System Architecture Diagram

Chapter 4: System Design

Systems Design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. It involves translating user requirements into a detailed blueprint that guides the implementation phase.

4.1 Basic Modules

The **Stock-er Inventory Management System** is architected around a modular design that ensures scalability, reusability, and maintainability of code and functionality. Each module encapsulates a specific aspect of system behaviour while enabling smooth integration with other components. Below are the key modules that constitute the core structure of the application:

1. User Authentication & Role Management Module

- Handles secure user login and registration workflows.
- Implements **role-based access control** for Admins, Managers, and Staff.
- Incorporates **two-factor authentication (2FA)** and credential encryption.
- Supports invitation-based user onboarding and password recovery.

2. Inventory Management Module

- Enables creation, editing, and deletion of inventory items.
- Maintains detailed product information including name, SKU, quantity, price, supplier info, and image attachments (up to 10 images per item).
- Provides **category-wise item organization** and advanced item filtering.
- Supports low-stock alerts and inventory valuation metrics.

3. Barcode & QR Code Integration Module

- Automates barcode generation for each inventory item.
- Enables three barcode-based lookup and checkout methods:
 - **USB HID barcode scanner support** (hardware).
 - **Manual barcode entry** (keyboard).
 - **Barcode image upload and decoding** (image recognition).
- Includes QR Code generation for extended compatibility and flexibility.

4. Billing & Checkout Module

- Facilitates item checkout with automatic cart population via barcode scanning.
- Applies tax rates and configurable discounts (flat or percentage).
- Generates and exports digital receipts using **iTextSharp** (PDF).
- Maintains transaction history for audit purposes.

5. Image Management Module

- Allows users to **upload, preview, and manage multiple images per item**.
- Supports drag-and-drop functionality for user convenience.
- Ensures optimized image handling and cloud-safe storage referencing.

6. Cloud Database Management Module

- Interfaces with **MongoDB Atlas** for cloud-hosted data access.
- Handles CRUD operations for users, inventory, billing, and logs.
- Supports real-time data synchronization across connected systems.

7. UI/UX Module

- Uses **Guna.UI2.WinForms** for a clean, responsive, and modern UI design.
- Supports dynamic layouts, smooth transitions, and themed controls.
- Adapts UI components based on user role and permissions.

8. System Logging & Audit Trail Module

- Tracks user activities such as login attempts, item changes, and transactions.
- Maintains audit logs to enhance system transparency and accountability.
- Helps administrators monitor system behaviour and respond to anomalies.

9. Settings & Configuration Module

- Allows administrators to define tax rates, discount schemes, and system preferences.
- Configures barcode settings, email templates, and UI themes.
- Supports localization and adaptable settings per organization (multi-tenant environments).

4.2 Data Design

The data design of the **Stock-er Inventory Management System** is structured around a document-based architecture using MongoDB Atlas, facilitating efficient storage, querying, and management of inventory-related data. The schema is optimized for extensibility and real-time cloud synchronization, allowing multi-organization support through logical data isolation and role-based access.

Each core entity within the system reflects real-world components such as users, inventory items, billing, and categorization. The schema design prioritizes scalability, performance, and ease of integration with the front-end WinForms application.

4.2.1 Core Data Entities and Attributes

1. User

Defines individuals registered in the system with role-based access.

- *Id* – Unique identifier (ObjectId)
- *Email* – User's email address
- *PasswordHash* – Encrypted password
- *Role* – Enum: *Admin* / *Manager* / *Staff* / *LoggedOut*
- *OrganizationId* – Foreign key to Organization
- *CreatedBy* – ID of the user who created this user
- *FirstLogin* – Boolean for first-time login detection
- *CreatedAt* – Account creation timestamp
- *SignupCode*, *SignupCodeExpiry* – Verification on organization sign-up
- *ResetPasswordCode*, *ResetPasswordCodeExpiry* – For password recovery
- *DeleteConfirmationCode*, *DeleteConfirmationCodeExpiry* – For account deletion
- *Status* – Account status (active, deleted, pending)

2. Organization

Represents a tenant (company or unit) within the system.

- *Id* – Unique identifier
- *Name* – Organization name
- *CreatedByUserId* – User who registered the organization
- *CreatedAt* – Timestamp of organization creation

3. Item

Defines inventory items stored by the organization.

- *Id* – Unique identifier
- *Name, Description* – Item name and description
- *CategoryId* – Link to item category
- *Quantity* – Available stock quantity
- *Price* – Unit price
- *OrganizationId* – Owning organization
- *Barcode* – Nested object with:
 - *Type* – Barcode type (e.g., Code128, EAN13)
 - *BarcodeString* – Encoded value
 - *BarcodeImage* – Image path or base64 string
- *Images* – List of up to 10 image paths

4. Category

Represents logical item groupings.

- *Id* – Unique identifier
- *Name* – Category name
- *ItemCount* – Count of items in the category
- *OrganizationId* – Owning organization

5. Bill

Captures itemized billing and transaction details.

- *Id* – Unique identifier
- *BillNumber* – Auto-generated bill identifier
- *OrganizationId* – Owning organization
- *BilledBy* – User who generated the bill
- *BillDate* – Timestamp of billing
- *Items* – List of BillItem Nested objects
 - *ItemId* – Reference to the sold item
 - *Barcode, ItemName* – Lookup references
 - *Quantity* – Sold quantity
 - *UnitPrice, TotalPrice* – Pricing per unit and total

- *SubTotal* – Pre-tax total
- *TaxRate, TaxAmount* – Tax calculation
- *DiscountType* – 0: Flat, 1: Percentage
- *DiscountValue, DiscountAmount* – Discount data
- *GrandTotal* – Final payable amount
- *BillPath* – Filepath to generated PDF bill
- *IsCheckedOut* – Boolean flag for transaction completion

6. TaxOption

Used to define applicable tax configurations.

- *Rate* – Tax percentage (e.g., 5.0, 18.0)
- *Display* – Label for UI display (e.g., "GST 18%")

7. DiscountType (Enum)

Enumerates discount modes used during billing.

- *Flat* – Fixed value off the subtotal
- *Percentage* – Percentage-based deduction

4.2.2 Schema Characteristics

- **Document-Oriented Storage:** Uses nested objects (like BarcodeData and BillItem) to maintain atomicity and reduce external joins.
- **Multi-Tenant Isolation:** Every data object includes OrganizationId to enforce secure, isolated access.
- **Barcode Compatibility:** Items are assigned barcode metadata compatible with USB HID scanners and image decoders.
- **PDF Path Storage:** Bills include the saved path to the locally generated PDF for later viewing or printing.

4.3 Procedural Design

The procedural design of the **Stock-er Inventory Management System** formalizes the workflows and logic that power each user interaction and system behavior. Each process follows a clearly defined path—from input validation to database operations and UI feedback—ensuring consistency, scalability, and alignment with business logic.

The system follows an **event-driven architecture**, with modules responding to user inputs and role-based events. MongoDB Atlas serves as the centralized cloud-based backend, and all operations are conducted within the multi-tenant context defined by the user's organization.

4.3.1 Core Workflows

The core workflows in Stock-er are:

1. Admin Registration and User Login Workflow

- **Admin Registration:**

- An admin user can initiate system use by signing up with a **new organization**.
- The system creates a new Organization entry.
- The registering user becomes the first admin and organization owner.

- **Invited User Signup:**

- Admins and Managers can invite users via email.
- The invite includes a **secure invite code**, valid for a limited time.
- The invitee enters the code at registration and is assigned their predefined role.

- **Login:**

- User enters email and password.
- System verifies the password hash and organization context.
- Upon successful login, the system loads the user's role and redirects them to the **Inventory Dashboard**.

2. Inventory Management Workflow

- Admins and Managers can create, edit, or delete inventory items.
- The system validates all required fields:
 - Name, Description, Category, Quantity, Price, Barcode data.
- Up to **10 item images** are supported:

- Each image is converted to **base64 format** and stored on the cloud.
- **Barcode Generation:**
 - A barcode is created using **BarcodeLib**, based on the selected type (e.g., Code128, EAN13).
 - The generated barcode image is also stored as **base64 in the cloud**.
- The finalized item is stored under the correct *OrganizationId* with timestamps.

3. Billing and Checkout Workflow

- Users initiate billing by scanning or selecting items.
- **Three methods** are supported:
 1. **Barcode Scanning** (USB HID devices)
 2. **Manual Barcode Entry**
 3. **Image Upload and Decode** (via **ZXing.Net**)
- Items are validated for availability and added to a virtual cart.
- System computes:
 - Subtotal
 - Discount (Flat or Percentage)
 - Tax (from TaxOption)
 - Grand Total

- A **PDF receipt** is generated using **iTextSharp**.
- The bill is:
 - Saved locally as PDF
 - Stored in the database with a reference (BillPath)
 - Inventory is updated accordingly

4. Barcode Processing Workflow

- **Generation:**
 - Uses **BarcodeLib** during item creation.
 - Barcode image and metadata are stored (base64-encoded) in the item record.
- **Decoding:**
 - At checkout or manual lookup, **ZXing.Net** processes:
 - HID Scanner input (treated as keyboard input)
 - Image uploads of barcodes

5. Role-Based Access Enforcement

The system strictly controls module and action access through role validation:

- **Admin:**
 - Full system control, including inviting users of any role and organization creation.

- **Manager:**

- Can manage inventory, checkout, and invite Managers/Staff.

- **Staff:**

- Can only perform billing and view inventory.

Interface elements are conditionally rendered based on role, ensuring clear boundaries between access levels.

6. Bill Lifecycle Handling

- Each checkout generates a new Bill object.
- Contains embedded BillItems, discount details, and tax calculations.
- The bill is associated with:
 - OrganizationId
 - BilledBy (User)
- PDF is saved locally; the file path is recorded in the database.

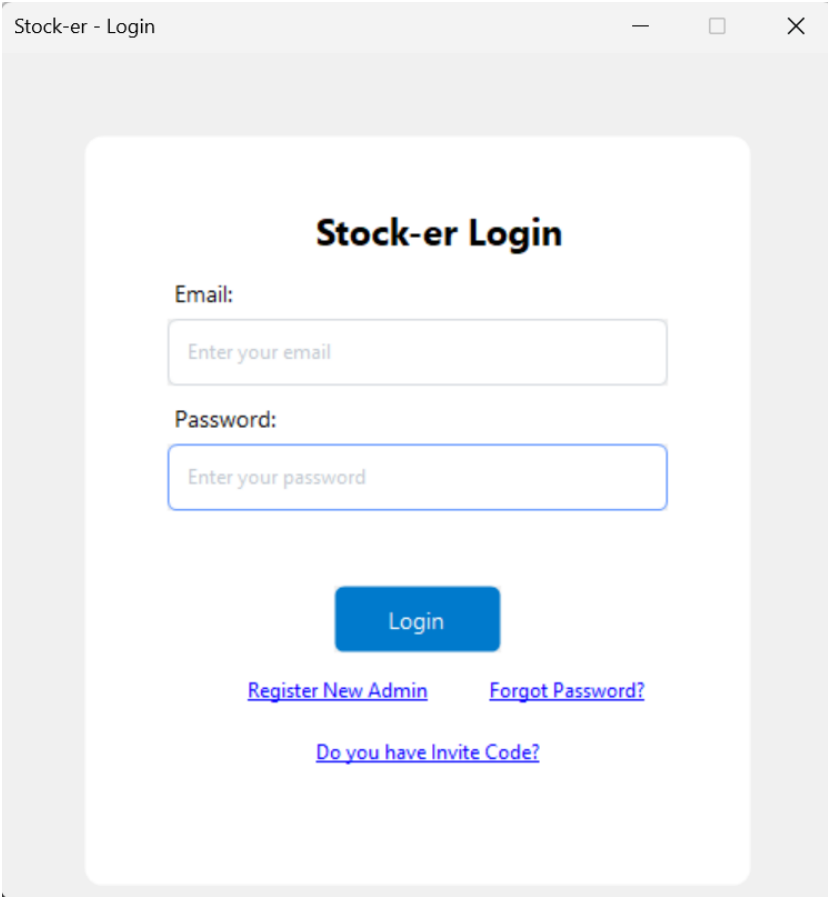
7. Image Upload (Item & Barcode)

- Users can attach up to **10 images per item**.
- Barcode image is also generated and stored.
- All images are converted to **base64 strings** and uploaded to a secure cloud location.
- No images are stored locally on the client machine after upload.

8. Audit Logging

- Although a structure exists for system activity logging, logs are **not yet implemented** in the current release.
- Intended for future auditing features to track operations like login, edits, deletions, and billing.

4.4 User Interface Design

A screenshot of a web browser window titled "Stock-er - Login". The page has a light gray background. In the center, there is a white rounded rectangle containing the login form. The form has the title "Stock-er Login" in bold black text. Below the title, there are two input fields: "Email:" with a placeholder "Enter your email" and "Password:" with a placeholder "Enter your password". Below these fields is a blue "Login" button. At the bottom of the form, there are three links: "Register New Admin", "Forgot Password?", and "Do you have Invite Code?".

Stock-er - Login

Stock-er Login

Email:
Enter your email

Password:
Enter your password

Login

[Register New Admin](#) [Forgot Password?](#)

[Do you have Invite Code?](#)

Figure 4.4.1 – Login Page 1

The screenshot shows a web browser window titled "Stock-er - Admin Registration". The main heading is "Complete Registration". Below the heading is a form with a label "Email" and a text input field containing the placeholder text "Enter your email". Below the input field is a blue button labeled "Continue with Invite".

Figure 4.4.2 Invitation Code Registration Page

The screenshot shows a web browser window titled "Stock-er - Admin Registration". The main heading is "Register New Admin". Below the heading is a form with a label "Email" and a text input field containing the placeholder text "Enter your email". Below the input field is a blue button labeled "Request Verification Code". At the bottom of the form, there is a link that says "Already have account? [Log in](#)".

Figure 4.4.3 Admin Registration Page

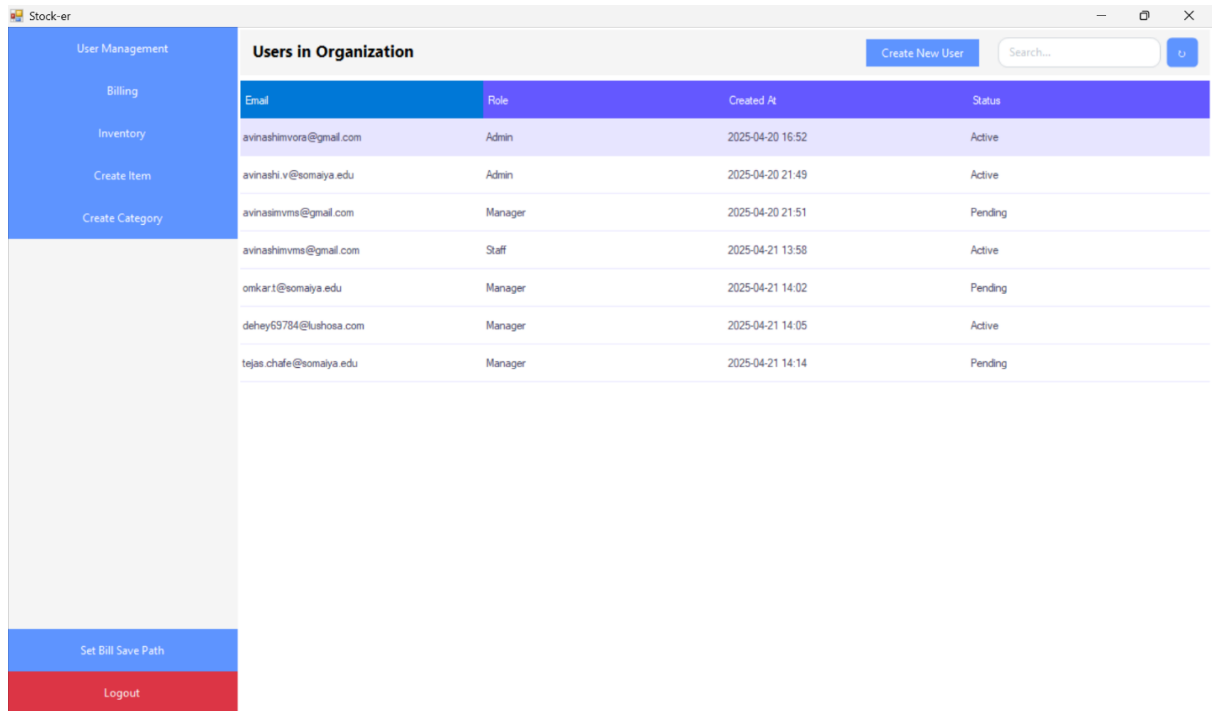


Figure 4.4.4 Dashboard

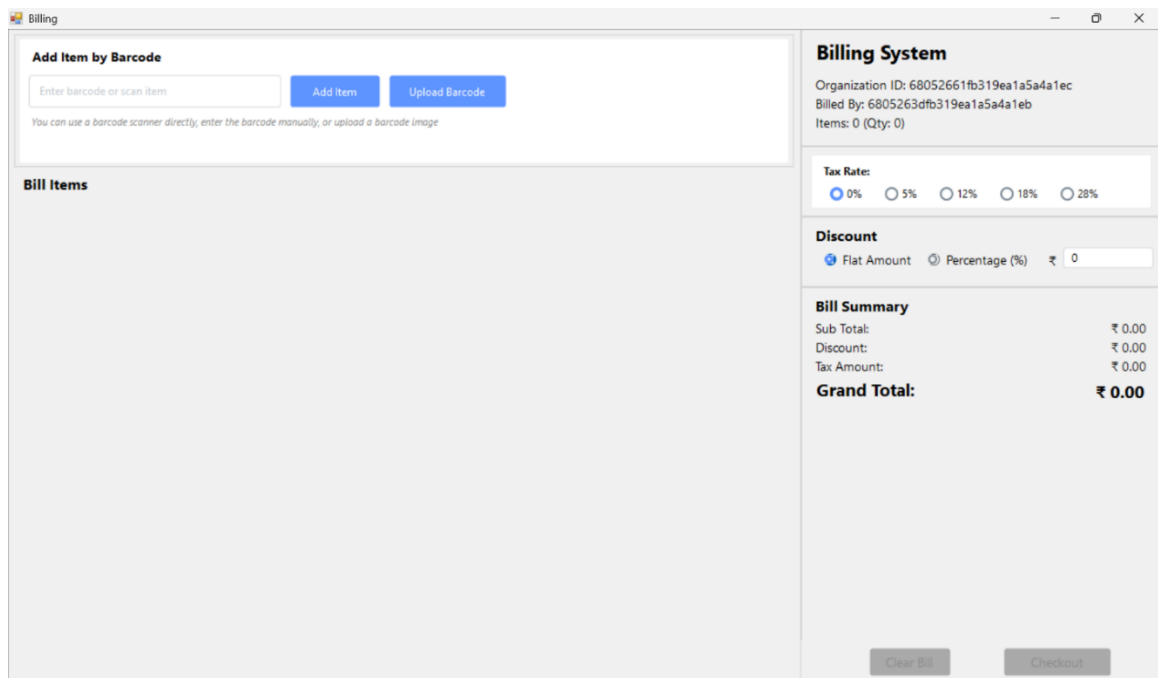


Figure 4.4.5 Billing Page

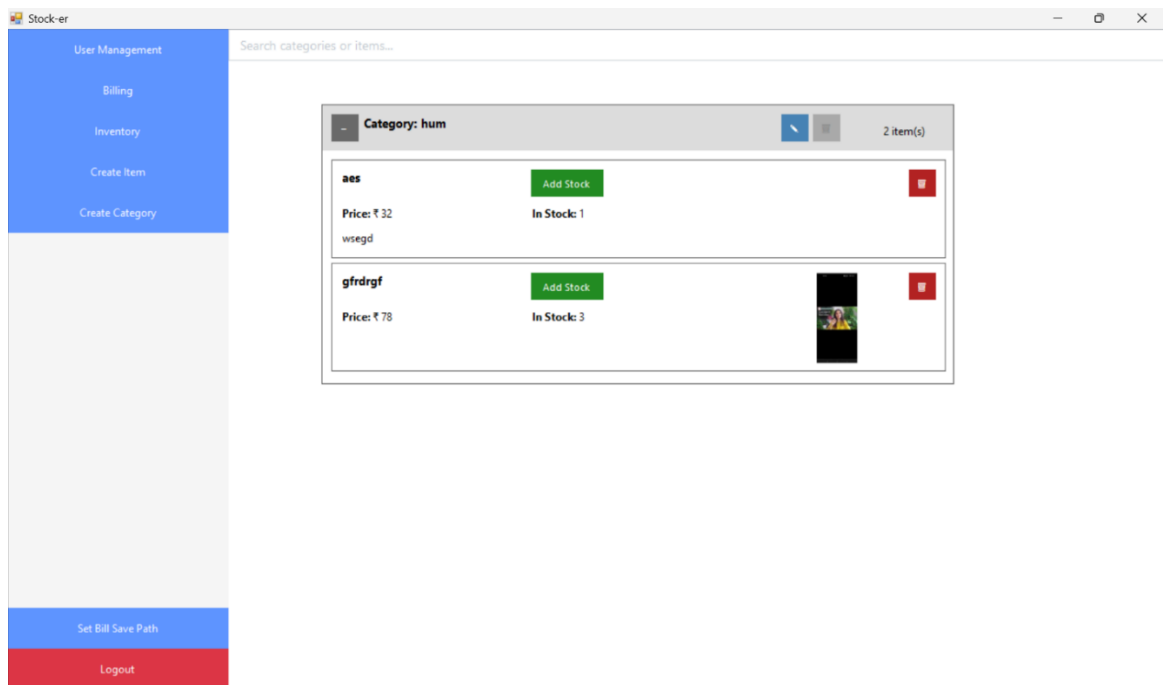


Figure 4.4.6 Manage Category Page

The 'Item' addition form contains the following elements:

- Item Name:** A text input field.
- Description (optional):** A text input field.
- Category:** A dropdown menu currently showing 'hum'.
- Price:** A text input field.
- Quantity:** A numeric input field currently showing '0'.
- Code93:** A dropdown menu.
- Image Upload Area:** A large rectangular box for uploading images, with left and right navigation arrows below it.
- Buttons:** 'Add Images', 'Delete Image', and a large 'Save' button.

Figure 4.4.7 Item Addition Page

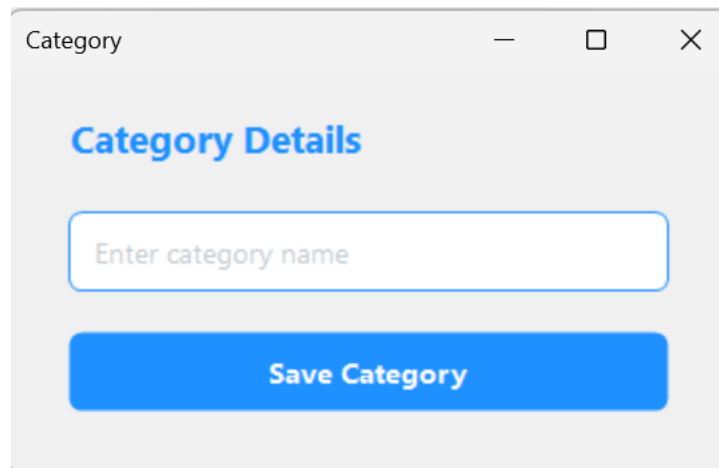


Figure 4.4.8 Category Details

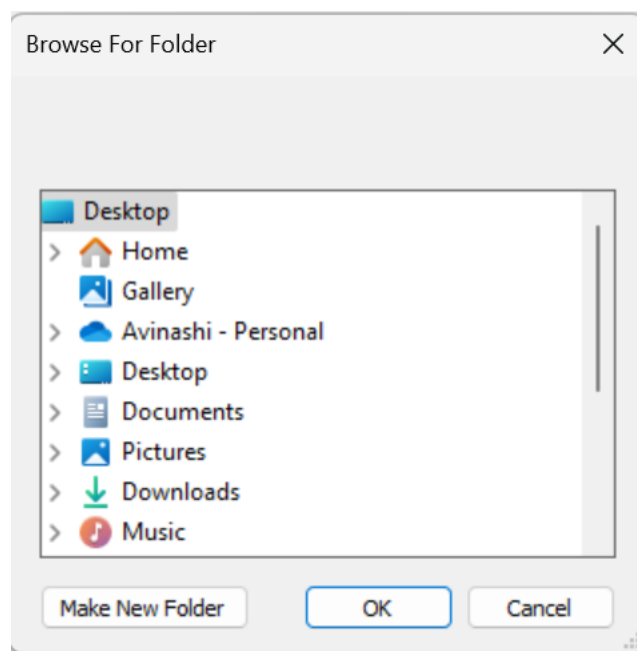


Figure 4.4.9 Browse to Add

4.5 Security Issues

Security is a critical aspect of any inventory management system, especially when handling sensitive data such as user credentials, organizational information, and billing records. The following outlines key security concerns identified in the development and deployment of the Stock-er system, along with implemented mitigation strategies.

1. Authentication and Authorization

- **Issue:** Weak or improper authentication mechanisms can lead to unauthorized access.
- **Mitigation:** Stock-er uses a secure login system with password hashing using BCrypt.Net-Next. Role-based access control (RBAC) ensures that Admins, Managers, and Staff have appropriate permissions based on their roles.

2. Data Storage and Access

- **Issue:** Sensitive data (e.g., user information, billing records) may be exposed if not properly protected in storage.
- **Mitigation:** All sensitive user data is stored securely in MongoDB with access controlled through scoped database operations. File paths for PDF bills are stored instead of the files themselves, which are securely stored on the file system with restricted access.

3. Data Transmission

- **Issue:** Interception of data during client-server communication can lead to data breaches.
- **Mitigation:** Though the current prototype is a desktop application, future deployments over a network will employ HTTPS/TLS encryption for secure data transmission.

4. Barcode and QR Input Security

- **Issue:** Malformed or malicious input via barcode/QR scanners or image uploads could exploit parsing mechanisms.
- **Mitigation:** All input is sanitized and validated before decoding with ZXing.Net. Uploaded images are checked for format and size constraints to avoid potential DoS (Denial of Service) via large or corrupted files.

5. PDF Generation Vulnerabilities

- **Issue:** The PDF generation process may be abused to inject scripts or malformed content.
- **Mitigation:** The iTextSharp library is used with predefined templates, ensuring that dynamic inputs are sanitized before being inserted into PDF documents.

6. Insider Threats

- **Issue:** Users with elevated privileges could abuse their access.
- **Mitigation:** Activity logging and user management interfaces are available to Admins and Managers for regular auditing. Role limitations restrict sensitive operations based on job function.

7. Dependency Risks

- **Issue:** Vulnerabilities in third-party packages could be exploited if not monitored.
- **Mitigation:** All dependencies are reviewed periodically. Only trusted, actively maintained libraries (e.g., Newtonsoft.Json, MongoDB.Driver) are used.

4.6 Test Case Design

Test cases for Stock-er were organized module-wise to ensure functional and security coverage across all critical operations.

1. Authentication Module

Test Case 1: Register with valid organization and admin details.

Test Case 2: Attempt registration with an already used email.

Test Case 3: Login with valid admin credentials.

Test Case 4: Login with invalid password.

Test Case 5: Access dashboard without login.

Test Case 6: Logout from top bar menu.

2. Inventory Management Module

Test Case 7: Add a new item with all required fields filled.

Test Case 8: Add item with missing category selection.

Test Case 9: Edit existing item details.

Test Case 10: Delete an item and verify removal from the database.

Test Case 11: Add a new category.

Test Case 12: Edit category name.

Test Case 13: Delete a category and check if orphan items are handled.

Test Case 14: Update stock quantity of an existing item.

Test Case 15: View items by category.

3. Billing Module

Test Case 16: Initiate billing with barcode string input.

Test Case 17: Initiate billing via barcode scanner input.

Test Case 18: Upload barcode image and auto-fill item data.

Test Case 19: Add multiple items to a single bill.

Test Case 20: Apply tax and discount to a bill.

Test Case 21: Download invoice as PDF.

Test Case 22: Store PDF path in database correctly.

Test Case 23: Generate bill without selecting any item (validation test).

4. User Management Module (Admin/Manager)

Test Case 24: Admin invites a new staff member.

Test Case 25: Manager edits role of an existing user.

Test Case 26: Attempt to access user management as staff (unauthorized access).

Test Case 27: Admin views all registered users.

Test Case 28: Delete a user and verify account deactivation.

5. General System Tests

Test Case 29: System handles empty barcode input gracefully.

Test Case 30: Attempt to upload a non-image file as barcode image.

Test Case 31: Handle corrupted image upload during billing.

Test Case 32: Simultaneous item edits by two roles.

Test Case 33: System remains stable after logging in and out repeatedly.

Chapter 5: Implementation and Testing

5.1 Implementation Approaches

The implementation of Stock-er followed a modular, role-oriented approach built around maintainability, scalability, and real-world business workflows. The application was developed as a Windows Forms desktop application with cloud-synced MongoDB Atlas as the backend database. Below is a summary of the key implementation strategies.

5.1.1 Layered Architecture

Stock-er was developed using a **three-layered architecture**:

- **Presentation Layer:** Provides an intuitive and responsive UI using Windows Forms and modern controls.
- **Business Logic Layer:** Implements all core operations including inventory actions, user role validations, billing logic, and security measures.
- **Data Access Layer:** Interfaces with MongoDB Atlas through structured access methods for fetching, inserting, and updating data.

This structure allows for separation of concerns, making testing and future expansion easier.

5.1.2 Component-driven Development

Stock-er's major functionalities are organized into independently developed and testable components:

- **Authentication and Authorization:** Secure login and registration flows using hashed credentials and role-based access.

- **Inventory Management:** Modular support for items, categories, and stock updates with CRUD operations.
- **Billing System:** Allows adding items via barcode string, scanner, or image upload; generates PDF invoices.
- **User Management:** Admins and Managers can add, update, and delete users with different permission scopes.
- **PDF Handling and Storage:** Billing receipts are generated as PDFs and stored locally, with file paths linked in the cloud database.

5.1.3 Cloud-connected Offline Workflow

While Stock-er operates as a desktop application, it maintains **real-time cloud sync with MongoDB Atlas**, ensuring data is accessible and backed up. This design offers the **benefit of cloud-based persistence** while retaining **local responsiveness and performance**.

5.1.4 Versioning and Maintenance Strategy

The implementation includes:

- **Version Control:** Git-based branching for feature isolation and collaborative development.
- **Development Workflow:** Features are built and tested incrementally to minimize integration conflicts and maintain stability across modules.

5.2 Coding Details and Coding Efficiency

The coding strategy for Stock-er emphasizes maintainability, performance, and clean separation of concerns. Through structured modularity and adherence to coding conventions, the project ensures scalability and ease of collaboration.

5.2.1 Code Organization

The project is organized into clearly defined folders, each representing a specific concern within the system:

- **Data/**
Handles MongoDB connection setup and configurations. Contains database client initialization and environment configuration for MongoDB Atlas.
- **Models/**
Contains C# class definitions representing entities such as Item, Category, User, StockRecord, and Bill. These models are used throughout the app for data binding and database interaction.
- **Services/**
Implements the business logic and data access layers. Key services include AuthService, BillingService, InventoryService, and UserService. Each service encapsulates operations like CRUD, validation, and role-checks.
- **UI/**
Houses all user interface components and logic. Subdivided into:
 - Forms/: Entry-level forms like LoginForm, MainForm, SignupForm.
 - Pages/: Embedded pages loaded dynamically in the main layout (InventoryPage, BillingPage, UsersPage).
 - Controls/: Reusable user controls like ItemCard, UserTile, SearchBox.

- **Utils/**

Contains utility classes and helper methods for common tasks such as image handling, PDF generation, barcode decoding, and general-purpose validation logic.

5.2.2 Naming Conventions

- Classes and methods use **PascalCase** (e.g., `GenerateInvoicePDF()`, `ValidateUserCredentials()`).
- Variables and method parameters follow **camelCase**.
- Constants are written in **UPPER_SNAKE_CASE**.
- Filenames directly reflect their content (e.g., `BillingService.cs`, `CategoryModel.cs`).

5.2.3 Efficient Coding Practices

- **Reusable services** reduce duplication, especially for MongoDB operations and validation logic.
- **Async/await patterns** are used for data-fetching methods to maintain UI responsiveness.
- **Form lifecycle** is managed cleanly: disposable forms and UI elements are properly disposed to avoid memory leaks.
- **Role-based UI rendering** avoids code branching by dynamically hiding/showing controls based on user roles.
- **Input validation** is handled via utility functions for reuse across multiple forms.

5.2.4 Performance Optimizations

- MongoDB queries are optimized with indexes on commonly queried fields like `ItemCode` and `Email`.

5.3 Testing Approaches

Testing is a critical component of the development cycle in Stock-er, ensuring the application's reliability, performance, and user experience. Below is a breakdown of the various testing approaches adopted during the development process:

1. Unit Testing:

Unit testing focuses on verifying the smallest units of functionality in isolation. In Stock-er, unit tests were crucial for validating core components such as:

- **Inventory Management:** Validating CRUD operations for items and categories.
- **Authentication Services:** Ensuring the correct validation of user credentials and role assignment.
- **Barcode Processing:** Verifying that barcode decoding logic via ZXing library works properly.
- **PDF Generation:** Ensuring that PDF bills are generated correctly with accurate tax and item data.

On the frontend, unit tests were used to ensure that UI validation, such as handling missing fields in forms (e.g., item name, category), behaves as expected. These tests help identify logic errors early, streamlining debugging.

2. Integration Testing:

Integration testing ensures that various modules in the system interact correctly. In Stock-er, integration testing focused on:

- **Authentication Flow:** Verifying that successful user login triggers the correct role-based redirection and access to respective dashboards.
- **Inventory Updates:** Testing interactions between the item management system (add/edit) and the MongoDB database.

- **Billing Process:** Ensuring smooth integration between barcode scanning, item fetching from the database, and PDF generation.

These tests confirm that multiple services, such as user authentication, inventory updates, and billing, work seamlessly together.

3. End-to-End (E2E) Testing:

E2E testing simulates real user behaviour from start to finish. Scenarios tested include:

- **Admin Flow:** Login → Manage Users → Edit Inventory → Generate Bill → Logout.
- **Manager Flow:** Login → Edit Categories → Create Bill → Logout.
- **Staff Flow:** Login → Scan Item → Add to Bill → Generate PDF → Logout.

These tests ensure that the entire user experience flows correctly across different roles, validating the critical interactions like login, inventory management, and billing work end-to-end without any errors.

4. Functional Testing:

Functional testing ensures that specific features work as required. In Stock-er, features tested include:

- **Item Search:** Verifying that searching by item code or category works correctly.
- **Inventory Edit:** Ensuring that edits to items (e.g., price or stock levels) are reflected accurately in the UI and database.
- **PDF Generation:** Confirming that the bill generation feature correctly reflects all items in the cart, applies taxes and discounts, and stores the resulting PDF file.

These tests ensure that individual features such as barcode scanning, item filtering, and PDF creation meet functional expectations.

5. Performance Testing:

Performance testing ensures the application performs efficiently under different loads. Key areas tested in Stock-er include:

- **MongoDB Query Performance:** Evaluating the responsiveness of MongoDB queries, especially for inventory lookups and item fetch operations.
- **PDF Generation Speed:** Verifying that bill generation is completed within an acceptable time frame, even with a large number of items in the cart.
- **UI Responsiveness:** Ensuring that the UI remains responsive when handling multiple items or large amounts of data.

These tests evaluate the system's ability to handle high traffic, large data sets, and time-consuming operations like generating PDFs or performing database lookups.

6. Security Testing:

Security testing ensures that sensitive data is protected and the system is resilient to attacks. In Stock-er, security testing focused on:

- **Authentication:** Verifying that unauthorized access is prevented and only authenticated users can access restricted routes.
- **Password Hashing:** Ensuring passwords are securely hashed using BCrypt.Net and never stored in plaintext.
- **Input Validation:** Testing for common vulnerabilities like SQL/NoSQL injection or XSS in form fields (e.g., item name or category fields).

Security testing ensures that all sensitive operations, like login and billing, are secure, and all user data is safely stored and transmitted.

7. Regression Testing:

Regression testing ensures that new updates or features do not break previously working

functionalities. In Stock-er, regression testing involved running a suite of automated tests every time new code was integrated. The tests were focused on verifying that:

- CRUD operations still function correctly after changes in the backend.
- User management features (like adding or removing users) remain stable after new feature implementations.
- Billing and PDF generation are unaffected by updates in inventory or database schema.

8. Usability Testing:

Usability testing evaluates how user-friendly the system is. While more informal, testing was done to ensure that:

- Users could easily navigate through different forms (e.g., Inventory, Billing, User Management) without confusion.
- Role-based restrictions (Admin vs. Staff) were clear, with intuitive UI elements indicating what users can and cannot do.
- Common tasks, like generating a bill or adding an item, were easy to accomplish without needing extensive guidance.

9. Compatibility Testing:

Stock-er is a Windows desktop application, and compatibility testing was done across different operating systems to ensure that:

- The application works on various versions of Windows (e.g., Windows 10, Windows 11).
- UI components like form sizes, buttons, and text fields render properly on different screen resolutions.
- The application integrates seamlessly with other tools, such as HID barcode scanners.

5.4 Modifications and Improvements

Throughout the development and testing phases of Stock-er, several modifications and improvements were made to enhance the system's functionality, user experience, and overall performance. These changes were driven by both initial design goals and user feedback, as well as insights gained during testing and real-world usage.

1. Feature Enhancements

- **User Role Management:** Initially, the role management system allowed only basic user authentication. However, based on user feedback, additional features were implemented to allow more granular permissions for different roles (Admin, Manager, and Staff). This modification ensures that only authorized users can perform sensitive tasks like managing users, editing categories, or generating billing reports.
- **Inventory Item Management:** To accommodate diverse business needs, the item management system was expanded. Initially, Stock-er allowed only basic item input and display. Later, functionality for bulk importing items via CSV and a more advanced search system (search by barcode, name, and category) were added, improving the system's usability and scalability.
- **Barcode Scanning Enhancements:** Barcode scanning was initially limited to manual input. Over time, this feature was extended to support multiple input methods, including manual string input, scanning via HID scanners, and image uploads, allowing for greater flexibility in inventory management.

2. Performance Improvements

- **Optimized Database Queries:** Early testing revealed performance bottlenecks when fetching large sets of inventory items. In response, several MongoDB queries were optimized, leveraging proper indexing and aggregation strategies. This significantly reduced loading times, especially for large inventories with thousands of items.

- **PDF Generation Speed:** Initially, PDF bill generation took longer with large orders. By optimizing the PDF generation process (e.g., reducing image resolution in the final output and using asynchronous processing for large files), the overall speed of bill generation was improved, resulting in a smoother user experience.

3. User Interface Improvements

- **Responsive Design Adjustments:** The initial design of the user interface was optimized for larger screens. However, following feedback from users working with smaller screen sizes, the UI was redesigned to ensure a more responsive layout. Controls, such as buttons and form fields, were adjusted for better usability across devices with different screen resolutions.
- **Error Handling and Notifications:** Initially, error handling was minimal, providing only basic error messages. After reviewing user feedback, error notification systems were enhanced, with more detailed messages and guidance on how to resolve issues (e.g., “Item not found” or “Invalid barcode format”). These improvements contributed to a more user-friendly experience, especially for users unfamiliar with the system.

4. Security Enhancements

- **Enhanced Password Security:** Initially, passwords were hashed using a simple hashing algorithm. However, after reviewing security best practices, the authentication system was upgraded to use bcrypt hashing (via the BCrypt.Net-Next library), ensuring that all user passwords are stored securely with high-strength encryption.
- **Session Management:** Initially, session management was simple, with no advanced features like token expiry. This was updated to include more robust session management, including token expiration and automatic logoff after periods of inactivity, thereby improving overall security.

5. Bug Fixes and Issue Resolutions

- **Barcode Scanning Compatibility:** An issue was identified where certain barcode scanners failed to scan certain product types. This was resolved by updating the ZXing barcode library and testing with a wider variety of scanners, ensuring compatibility with most commercially available devices.
- **PDF Formatting Errors:** Initially, some PDF bills had formatting issues (e.g., overlapping text or misaligned items). These bugs were addressed by refining the PDF template and testing with a variety of product configurations to ensure proper layout for all types of items.

6. Future Improvements

While Stock-er is fully functional and meets current business needs, several areas have been identified for future enhancement:

- **Mobile App Support:** A potential future improvement is the development of a mobile version of Stock-er, allowing users to manage inventory and generate bills directly from their smartphones or tablets.
- **Cloud Syncing for Inventory:** Although the database is hosted on MongoDB Atlas, further improvements could be made to support real-time syncing of inventory across multiple devices, enabling seamless access to up-to-date stock levels, especially for businesses operating in multiple locations.
- **Enhanced Reporting and Analytics:** Adding more advanced reporting features, such as trend analysis, sales forecasting, and stock alerts, would improve the system's ability to support decision-making.

Chapter 6: Result and Discussion

6.1 Test Report

Unit Testing

- **Test Case 1:** The unit test verifies that the JWT token is correctly created with the user's `_id` and `JWT_SECRET`. The generated token is checked to ensure it is valid and includes the necessary payload for authentication.
- **Test Case 2:** This unit test checks the logic of validating user credentials during login. It ensures that the correct user is authenticated when provided with the correct password.
- **Test Case 3:** The unit test validates the barcode decoding functionality. It checks that the ZXing library correctly decodes various barcode formats and returns accurate results.
- **Test Case 4:** This unit test verifies that the system correctly calculates the tax on the total bill based on the selected tax rate. It ensures the tax is added to the final amount.
- **Test Case 5:** This unit test checks the logic of updating the payment status in the user database after successful payment. The test ensures the `paymentStatus` field is set to `true` and properly updated in the user's record.

Integration Testing

- **Test Case 1:** This integration test verifies that the login flow works as expected. It ensures that a user can log in and be redirected to their dashboard after successful authentication.
- **Test Case 2:** This integration test checks the interaction between the inventory management module and the database. It verifies that new inventory items can be added, updated, and retrieved correctly from MongoDB.

- **Test Case 3:** This integration test ensures that the billing process integrates smoothly with the barcode input. It verifies that the system fetches the correct item from the database after scanning a barcode.
- **Test Case 4:** This test checks if the PDF generation process interacts correctly with the inventory data. It ensures that the items, tax, and total are accurately reflected in the generated PDF receipt.

End-to-End (E2E) Testing

- **Test Case 1:** This E2E test simulates the full workflow of an Admin user: Login → Manage Users → Edit Inventory → Generate Bill → Logout. It ensures that the Admin can complete all tasks without encountering errors.
- **Test Case 2:** This E2E test simulates the flow for a Staff user: Login → Scan Item → Add to Bill → Generate PDF → Logout. It ensures that the Staff user can perform their tasks efficiently.
- **Test Case 3:** This test verifies the Manager's workflow: Login → Edit Categories → Create Bill → Logout. It ensures that Managers can manage categories and create bills without any issues.

Functional Testing

- **Test Case 1:** This functional test checks the search functionality for inventory items. It verifies that users can search by barcode, item name, or category and retrieve relevant results.
- **Test Case 2:** This functional test ensures that the item editing functionality works as expected. It verifies that users can modify item details (e.g., price, quantity) and the changes are reflected in the database.

- **Test Case 3:** This test validates the PDF bill generation functionality. It ensures that when an order is processed, the bill is generated with all item details, taxes, and discounts correctly included.

Performance Testing

- **Test Case 1:** This performance test checks the speed of database queries when fetching a large number of inventory items. It ensures that the system can handle high volumes of data without noticeable lag.
- **Test Case 2:** This performance test measures the time taken to generate a PDF bill for orders with multiple items. It ensures that PDF generation remains efficient even with large orders.
- **Test Case 3:** This test verifies the system's responsiveness when multiple users are accessing the application simultaneously. It ensures the system can handle concurrent users without degradation in performance.

Security Testing

- **Test Case 1:** This security test ensures that user passwords are securely hashed using bcrypt. It checks that the system does not store plaintext passwords.
- **Test Case 2:** This test checks if the authentication system properly protects sensitive routes. It ensures that users without a valid token cannot access restricted areas of the application.
- **Test Case 3:** This test verifies input validation to prevent SQL/NoSQL injection attacks. It ensures that user inputs are sanitized and malicious inputs are rejected.

Regression Testing

- **Test Case 1:** This regression test checks if the inventory search functionality still works as expected after recent code updates. It ensures no breakage in previously working features.
- **Test Case 2:** This test ensures that the billing process continues to function as expected after recent backend optimizations.
- **Test Case 3:** This regression test verifies that the user role management system continues to function correctly after updates to the authentication flow.

Usability Testing

- **Test Case 1:** This usability test verifies that users can easily navigate between inventory management, billing, and user profile pages. It ensures the system's navigation is intuitive and smooth.
- **Test Case 2:** This test checks the clarity of role-based access messages (e.g., error messages for unauthorized actions). It ensures that users receive clear feedback when attempting restricted actions.

Compatibility Testing

- **Test Case 1:** This compatibility test ensures that the system's UI renders correctly on different browsers (Chrome, Firefox, Edge). It checks for consistent appearance and functionality across browsers.
- **Test Case 2:** This test verifies that the application is responsive and adapts to various screen sizes, including mobile devices and tablets.

6.2 User Documentation

This guide will help you navigate through the features and functionalities of the platform, ensuring you can make the most out of its capabilities. Whether you are an Admin, Manager, or Staff member, Stock-er offers an intuitive and efficient interface to manage inventory, process sales, and handle user roles.

1. Getting Started

Creating an Account

- **Admin Sign-Up:** If you are an Admin, you can create an account by providing organization details, such as your company name, email address, and a secure password.
- **User Registration:** Users invited by the Admin can sign up using an invitation code. After entering the code, they will be prompted to complete their registration.

Logging In

- Use your email address and password to log in to the system. Admins, Managers, and Staff will have access based on their respective roles.

User Roles and Permissions

- **Admin:** Has full access to all features, including managing users and configuring settings.
- **Manager:** Can manage inventory and users but does not have the same level of access to system configurations as an Admin.
- **Staff:** Limited to managing inventory and generating bills, without access to user management.

2. Inventory Management

Viewing Inventory Items

- Navigate to the inventory dashboard to see all the available items in your stock. You can filter or search for specific items based on their name, category, or barcode.

Adding New Items

- Admins and Managers can add new items to the system by entering item details such as name, description, price, and quantity.

Editing and Deleting Items

- You can edit the details of existing items (e.g., price, stock level, description) or delete items from the inventory as needed. This functionality is available to Admins, Managers, and Staff.

Categorizing Items

- Items can be organized into categories to make them easier to find and manage. Categories can be created, edited, and deleted by Admins and Managers.

Managing Item Stock Levels

- Stock levels for each item can be updated manually or automatically through the sales process. This ensures that the system reflects real-time stock availability.

3. Billing and Checkout

Scanning and Adding Items to Cart

- Items can be added to the cart using barcode scanning or manual entry of barcode strings. The system will fetch the item details and add it to the checkout list.

Applying Discounts and Taxes

- Discounts and taxes can be applied to the cart total before generating the final bill. The system supports both fixed and percentage-based discounts.

Generating Bills and Invoices (PDF)

- Once the transaction is complete, a PDF bill can be generated and downloaded. This bill will include all items, taxes, discounts, and the final amount.

Payment Processing

- Payments can be processed either through an integrated payment gateway or manually by selecting payment status. After successful payment, the payment status will be updated.

4. User Management (Admin/Manager only)

Viewing and Managing Users

- Admins and Managers can view all users in the system and perform actions like editing user information or removing users as needed.

Assigning Roles and Permissions

- Admins can assign different roles (Admin, Manager, Staff) to users, granting them access to different features and areas of the system.

5. Reports and Analytics

Downloading Bills

- Reports can be exported and downloaded in PDF or CSV formats for offline analysis and record-keeping.

6. Security and Privacy

Password Management

- Users can reset their passwords through the “Forgot Password” link or by changing their password in the profile settings.

Two-Factor Authentication (2FA)

- To enhance security, two-factor authentication (2FA) can be enabled. This requires users to verify their identity through a secondary method, such as a code sent to their email.

Logout

- Users can log out from the system anytime by clicking on the logout option in the navigation bar.

Chapter 7: Conclusion

7.1 Summary

The Stock-er inventory management system provides a comprehensive and intuitive solution for managing inventory, processing sales, and overseeing user roles within an organization. By integrating features such as barcode scanning, category management, billing and invoice generation, the platform streamlines the tasks of managing and maintaining inventory data while ensuring seamless operations for different user roles.

The system supports flexible user roles, including Admin, Manager, and Staff, with each role having defined access levels and permissions. This ensures that sensitive data and settings are secure while providing users with the tools they need to perform their tasks efficiently. Additionally, the integration with MongoDB Atlas ensures a cloud-synced and scalable database solution, offering real-time updates and reliability for inventory management.

Key functionalities, including the ability to add, edit, and categorize inventory items, generate detailed PDF invoices, and perform secure payment processing, make Stock-er a versatile tool for any organization requiring efficient inventory and sales management.

In terms of security, the system implements robust measures, including secure authentication via JWT tokens and input validation, ensuring user data and operations are safe from unauthorized access and malicious attacks.

Overall, Stock-er is designed to help businesses streamline their inventory management processes, improve accuracy, and enhance the overall efficiency of operations, all while ensuring a user-friendly experience for staff at various levels of access. The platform's performance, scalability, and security features make it a reliable choice for managing large inventories and processing transactions securely.

7.2 Limitations of the System

While Stock-er provides a robust platform for inventory and sales management, there are some limitations that could affect its functionality and future scalability. These limitations include:

1. No Item-Wise Taxation:

- Currently, the system applies a single tax rate to the entire bill, rather than allowing for item-specific taxation. This can be restrictive for businesses that require different tax rates for individual products or services based on local tax regulations or product categories.

2. No Customer Database:

- Stock-er lacks a dedicated customer database, which means there is no built-in functionality to track customer-specific details such as purchase history, contact information, or preferences. This could limit the ability to offer personalized services, loyalty programs, or detailed customer insights.

3. Payment Methods Integration:

- The system does not yet support the integration of external payment gateways or a variety of payment methods. As a result, users are required to manually update the payment status, making the process less seamless and more prone to errors.

4. Limited Reporting and Analytics:

- Although Stock-er offers basic reporting features, advanced analytics such as predictive sales analysis, customer insights, and detailed financial metrics are not included. This limits the ability for businesses to make data-driven decisions or forecast inventory needs accurately.

5. No Multi-Location Support:

- Stock-er does not support the management of inventory across multiple locations. Businesses with multiple stores or warehouses must manage them separately, which could lead to inefficiencies or errors in tracking stock levels.

6. Basic User Roles and Permissions:

- While the system supports different user roles (Admin, Manager, and Staff), the permissions are relatively basic. Future enhancements could provide more granular control over user access, such as role-based restrictions for specific actions or settings within the system.

7. No Real-Time Inventory Syncing:

- Stock levels are updated only after actions such as billing or stock adjustments are made. Real-time syncing across multiple users or devices is not supported, which may result in discrepancies in stock data when multiple people are accessing the system simultaneously.

These limitations highlight areas where the system could be improved in future versions to meet evolving user needs and offer more flexibility for businesses with complex requirements. However, even with these limitations, Stock-er remains a powerful and efficient tool for inventory and sales management.

7.3 Future Scope

Stock-er has the potential for further enhancement and expansion to address its current limitations and cater to the evolving needs of businesses. The following are some potential areas for future development and improvement:

1. Item-Wise Taxation:

- Future versions of Stock-er could include the ability to apply different tax rates for individual items within a bill. This would allow businesses to comply with varying tax

regulations for different product categories and improve the accuracy of financial transactions.

2. Customer Database and CRM Integration:

- Adding a customer database would allow businesses to store customer information, track purchase history, and offer personalized services. Integration with a Customer Relationship Management (CRM) system could help improve customer engagement, loyalty programs, and targeted marketing efforts.

3. Payment Methods Integration:

- Stock-er could integrate with various payment gateways (such as Stripe, PayPal, or local payment providers) to allow for seamless, automated payment processing. This would enable users to directly process payments within the system, reducing manual intervention and increasing transaction efficiency.

4. Advanced Reporting and Analytics:

- The future version could include more advanced reporting and analytics capabilities, such as real-time sales tracking, inventory forecasting, customer behavior analysis, and financial performance metrics. This would provide businesses with deeper insights to make data-driven decisions and optimize operations.

5. Multi-Location Support:

- Support for managing inventory across multiple locations, such as different stores or warehouses, would enable businesses with a broader geographical presence to track and manage their inventory more effectively in real time.

6. Granular User Roles and Permissions:

- Enhancing the user role and permission system would provide more flexibility in managing access and actions within the platform. Granular control over specific features,

such as allowing certain staff members to only manage stock without access to billing, would improve security and operational efficiency.

7. Real-Time Inventory Syncing:

- Implementing real-time syncing of inventory data across multiple devices and users would ensure that stock levels are always up to date. This would be especially useful in businesses with high transaction volumes or multiple staff members accessing the system simultaneously.

8. Mobile App Integration:

- Developing a mobile app for Stock-er could allow users to manage inventory and sales on-the-go. This would be particularly beneficial for businesses with field agents or those needing access to the system outside the office environment.

9. Cloud-Based Features and Scalability:

- Expanding Stock-er's cloud capabilities could make the system even more scalable, offering automatic updates, data backups, and high availability. This would allow businesses to scale their operations without worrying about server maintenance or performance issues.

10. Integration with External Systems:

- Stock-er could be enhanced by integrating with other business tools and platforms such as ERP systems, accounting software, and e-commerce platforms (e.g., Shopify, WooCommerce). This would help businesses automate workflows and reduce the need for manual data entry.

By focusing on these areas, Stock-er can evolve into an even more powerful and versatile solution for businesses of all sizes, improving inventory management, enhancing customer experience, and driving operational efficiency.

References

Enterprise Resource Planning (ERP) Systems – Historical and Comparative Analysis

- Monk, E., & Wagner, B. (2012). *Concepts in Enterprise Resource Planning*. Cengage Learning.
- Klaus, H., Rosemann, M., & Gable, G. G. (2020). What is ERP? – A review of ERP research and practice. *Business Information Systems Engineering*, 62(2), 215-228.

Cloud Computing in Inventory Management – Basis for Modern Multi-Tenant, Real-Time Sync Features

- Armbrust, M., et al. (2010). A View of Cloud Computing. *Communications of the ACM*, 53(4), 50–58.
- Zhang, Z., et al. (2018). Cloud-based smart inventory management system. *Journal of Manufacturing Systems*, 49, 58-67.

Barcode Standards – Stock-er Supports Various Barcode Formats

- GS1 – <https://www.gs1.org/standards/barcodes>.
- GS1. (2023). *Global Standards for Barcodes – A Complete Guide*.

SaaS vs ERP in SMEs – Comparative Foundation for Positioning Stock-er

- Benlian, A., Hess, T., & Buxmann, P. (2009). Drivers of SaaS-Adoption – An Empirical Study of Different Application Types. *Business & Information Systems Engineering*, 1(5), 357–369.
- Kaila, R., & Sandvik, K. (2020). Cloud-based ERP adoption in small and medium enterprises: An empirical study. *Journal of Small Business and Enterprise Development*, 27(3), 419-438.

Cloud-Native Systems in Enterprise Software – For Scalability and Multi-Tenancy

- Villamizar, M., et al. (2015). Infrastructure Cost Comparison of Running Web Applications in the Cloud Using Azure and Monolithic and Microservice Architectures. *Proceedings of the 16th IEEE/ACM International Symposium on Cluster, Cloud, and Grid Computing*.
- McCool, M., et al. (2022). Cloud-native architectures: A comprehensive guide to cloud-native software design. *ACM Computing Surveys*, 54(6), 1-34.